

ГЛАВА 18

ИНСТРУМЕНТЫ ДЛЯ КРУПНОМАСШТАБНЫХ ПРОГРАММ

В этой главе...

18.1. Обработка исключений	966
18.2. Пространства имен	981
18.3. Множественное и виртуальное наследование	1002
Резюме	1018
Термины	1018

Язык C++ используется для решения проблем любой сложности — как незначительных, которые способен решить один программист за несколько часов вечером после основной работы, так и чудовищно сложных, требующих десятков миллионов строк кода и модифицируемых впоследствии на протяжении многих лет. Средства, описанные в предыдущих разделах этой книги, полезны для решения весьма широкого диапазона вопросов программирования.

Язык предоставляет некоторые средства, которые полезней в больших и сложных системах, чем в простых. Это средства обработки исключений, пространства имен и множественное наследование, являющиеся темой данной главы.

Крупномасштабное программирование предъявляет к языку более высокие требования чем те, которых достаточно для небольших групп разработчиков. К этим требованиям относятся следующие.

- Способность обрабатывать ошибки при помощи независимой подсистемы.
- Способность использовать библиотеки, разработанные более или менее независимо.
- Способность моделировать более сложные прикладные концепции.

В данной главе рассматриваются три предназначенных для этого средства языка C++: обработка исключений, пространства имен и множественное наследование.

18.1. Обработка исключений

Обработка исключений (exception handling) позволяет независимо разработанным частям программы взаимодействовать и решать проблемы, возникающие во время выполнения. Исключения позволяют отделять код обнаружения проблемы от кода ее решения. Часть программы, ответственная за обнаружение проблемы, может передать информацию о возникшей ситуации другой части программы, которая специально предназначена для решения подобных проблем.

Основы концепции применения исключений в языке C++ представлены в разделе 5.6 (стр. 257). В данном разделе эта тема рассматривается подробнее. Эффективное использование обработки исключений требует понимания происходящего при передаче исключения, его обработки и смысла объектов, сообщающих о том, что пошло не так.

18.1.1. Передача исключений

В языке C++ исключение *передается* (raise) *выражением* throw (передача исключения). Тип выражения throw, вместе с текущей цепочкой вызова, определяет, какой *обработчик* (handler) будет обрабатывать исключение. Выбирается ближайший обработчик в цепочке вызовов, соответствующий типу переданного объекта. Тип и содержимое этого объекта позволяют передающей части программы сообщать обрабатывающей части о том, что пошло не так.

Когда выполняется оператор throw, расположенные после него выражения игнорируются. Оператор throw передает управление соответствующему блоку catch. Блок catch может быть локальным для той же функции или функции, непосредственно или косвенно вызвавшей ту, в которой произошла ошибка, приведшая к передаче исключения. Тот факт, что управление передается из одного места в другое, имеет два важных следствия.

- Функции можно преждевременно покидать по цепочке вызовов.
- По достижении обработчика созданные цепочкой вызова объекты будут уничтожены.

Поскольку операторы после оператора throw не выполняются, он похож на оператор return: он обычно является частью условного оператора или последним (или единственным) оператором функции.

Прокрутка стека

При передаче исключения выполнение текущей функции приостанавливается и начинается поиск соответствующей директивы catch. Поиск начинается с проверки того, расположен ли оператор throw непосредственно в блоке try (try block). Если это так, проверяется соответствие переданного объекта

одному из обработчиков того блока `catch`, с которым связан данный блок `try`. Если соответствие в блоке `catch` найдено, исключение обрабатывается. В противном случае осуществляется выход из текущей функции, ее память освобождается, а локальные объекты удаляются. Затем поиск продолжается в вызывающей функции.

Если обращение к передавшей исключение функции находится в блоке `try`, проверяются обработчики того блока `catch`, который связан с ним. Если соответствие найдено, исключение обрабатывается. В противном случае осуществляется выход и из вызывающей функции, а поиск продолжается в той функции, которая вызвала ее, и так далее.

Этот процесс, известный как *прокрутка стека* (*stack unwinding*), продолжается по цепи обращений вложенных функций до тех пор, пока не будет найден соответствующий исключению обработчик `catch`, а если он найден не будет, то до конца функции `main()`.

Как только способный обрабатывать исключение блок `catch` будет найден, выполнение продолжится в этом обработчике. По завершении работы обработчика выполнение продолжится с точки, расположенной непосредственно после последней директивы блока `catch`.

Если соответствующий блок `catch` не найден, программа завершает работу. Исключения предназначены для событий, препятствующих нормальному продолжению выполнения программы. Поэтому переданное исключение не может остаться необработанным. Если соответствующий блок `catch` не найден, программа вызывает библиотечную функцию `terminate()`, которая прекращает выполнение программы.



Необработанное исключение завершает программу.

Объекты автоматически удаляются при прокрутке стека

В ходе прокрутки стека происходит преждевременный выход из функции, содержащей оператор `throw`, а возможно, и из других функций по цепи обращений. Как правило, функции создают локальные объекты, которые при выходе из функции удаляются. При выходе из функции в связи с передачей исключения компилятор гарантирует правильное удаление локальных объектов. Когда завершается работа любой функции, ее локальное хранилище освобождается. Перед освобождением памяти удаляются все локальные объекты, которые были созданы до передачи исключения. Если локальный объект имеет тип класса, для него автоматически вызывается деструктор. Как обычно, для удаления объектов встроенного типа компилятор ничего не делает.

Если исключение происходит в конструкторе, значит, объект находится еще на стадии создания и может быть закончен только частично. Некоторые

из его членов, возможно, уже инициализированы, а другие, возможно, нет. Даже если объект создан только частично, следует гарантировать корректное удаление составляющих его членов.

Точно так же исключение могло бы произойти во время инициализации элементов массива или контейнера библиотечного типа. Корректное удаление элементов, созданных прежде, чем произошло исключение, также следует гарантировать.

Деструкторы и исключения

Тот факт, что деструктор запущен, но код в функции, освобождающий ресурс, может быть пропущен, влияет на структуру создаваемых программ. Как упоминалось в разделе 12.1.4 (стр. 595), если блок резервирует ресурс, а исключение происходит перед кодом, который его освобождает, освобождающий ресурс код не будет выполнен. С другой стороны, ресурсы, распределенные объектом класса, обычно освобождаются их деструктором. Использование классов для контроля резервирования ресурсов гарантирует правильность их освобождения, если функция завершается нормально или в результате исключения.

Факт запуска деструктора во время прокрутки стека влияет на то, как следует создавать деструкторы. Во время прокрутки стека исключение уже передано, но еще не обработано. Если во время прокрутки стека передается новое исключение и не обрабатывается в передавшей его функции, то вызывается функция `terminate()`. Поскольку деструкторы могут быть вызваны во время прокрутки стека, они никогда не должны передавать исключений, которые не обрабатывает сам деструктор. Таким образом, если деструктор выполняет операцию, которая могла бы передать исключение, он должен заключить ее в блок `try` и обработать локально в деструкторе.

На практике, поскольку деструкторы освобождают ресурсы, маловероятно, что они передадут исключения. Все типы стандартной библиотеки гарантируют, что их деструкторы не будут передавать исключение.



Во время прокрутки стека для локальных объектов классов выполняются деструкторы. Поскольку деструкторы выполняются автоматически, они не должны передавать исключений. Если во время прокрутки стека деструктор передаст исключение, которое он не обрабатывает, то программа будет завершена.

Объект исключения

Компилятор использует выражения передачи исключения для инициализации копией (см. раздел 13.1.1, стр. 631) специального объекта известного как *объект исключения* (exception object). В результате, у выражения в блоке `throw` должен быть полный тип (см. раздел 7.3.3, стр. 362). Кроме того, если у выражения тип класса, то его деструктор, конструктор копий и конструктор переме-

жения должны быть доступны. Если выражение имеет тип массива или функции, выражение преобразовывается в соответствующий ему тип указателя.

Объект исключения располагается в управляемой компилятором области памяти, которая будет гарантировано доступна для любого обработчика. Объект исключения удаляется после того, как исключение будет полностью обработано.

Как уже упоминалось, при передаче исключения осуществляется выход из всех блоков по цепочке вызовов, пока не будет найден соответствующий обработчик. При выходе из блока вся память, используемая его локальными объектами, освобождается. В результате передача указателя на локальный объект почти наверняка будет ошибкой. Причина этой ошибки та же, что и у ошибки возвращения из функции указателя на локальный объект (см. раздел 6.3.2, стр. 295). Если указатель указывает на объект в блоке, выход из которого осуществляется перед обработчиком, то этот локальный объект будет удален до обработчика.

При передаче исключения его выражение определяет статический тип (тип времени компиляции) (см. раздел 15.2.3, стр. 759) объекта исключения. Этот момент важно иметь в виду, поскольку большинство приложений передают исключения, тип которых исходит из иерархии наследования. Если выражение `throw` обращается к значению указателя на тип базового класса и этот указатель указывает на объект производного класса, то переданный объект отсекается (см. раздел 15.2.3, стр. 762) и передается только часть базового класса.



Передача указателя требует, чтобы объект, на который указывает указатель, существовал на момент выполнения соответствующего обработчика.

Упражнения раздела 18.1

Упражнение 18.1. Каков тип объекта исключения в следующих операторах `throw`?

```
(a) range_error r("error");    (b) exception *p = &r;
    throw r;                    throw *p;
```

Что было бы, будь оператор `throw` в случае (b) написан как `throw p`?

Упражнение 18.2. Объясните, что случится, если исключение произойдет в указанном месте:

```
void exercise(int *b, int *e)
{
    vector<int> v(b, e);
    int *p = new int[v.size()];
    ifstream in("ints");
    // исключение происходит здесь
}
```

Упражнение 18.3. Существуют два способа исправить предыдущий код. Опишите и реализуйте их.

18.1.2. Обработка исключения

Объявление исключения (exception declaration) в директиве `catch` (catch clause) выглядит как список параметров функции, только с одним параметром. Как и в списке параметров, имя параметра обработчика можно пропустить, если у блока `catch` нет необходимости в доступе к переданному исключению.

Тип объявления определяет виды исключений, обрабатываемых обработчиком. Тип должен быть завершенным (см. раздел 7.3.3, стр. 362). Тип может быть ссылкой на l-значение, но не ссылкой на r-значение (см. раздел 13.6.1, стр. 673).

При входе в блок `catch` параметр в объявлении исключения инициализируется объектом исключения. Подобно параметру функции, если тип параметра обработчика не является ссылочным, параметр обработчика копирует объект исключения; изменения, внесенные в параметр в обработчике, осуществляются с его локальной копией, а не с самим объектом исключения. Если параметр имеет ссылочный тип, то, как любой ссылочный параметр, параметр обработчика будет только другим именем объекта исключения. Изменения, внесенные в ссылочный параметр, осуществляются с самим объектом исключения.

Подобно объявлению параметра функции, параметр обработчика, имеющий тип базового класса, может быть инициализирован объектом исключения типа производного класса. Если у параметра обработчика будет не ссылочный тип, то объект исключения будет отсечен (см. раздел 15.2.3, стр. 762), как и при передаче такого объекта обычной функции по значению. С другой стороны, если параметр является ссылкой на тип базового класса, то параметр будет связан с объектом исключения обычным способом.

Также, подобно параметрам функции, статический тип объявления исключения определяет действия, которые может выполнить обработчик. Если у параметра обработчика будет тип базового класса, то обработчик не сможет использовать члены, определенные в производном классе.

Рекомендуем

Обычно обработчики, получающие исключения типа, связанного наследственными отношениями, определяют свой параметр как ссылку.

Поиск соответствующего обработчика

Блок `catch`, найденный в ходе поиска соответствующего обработчика, не обязательно является наиболее подходящим данному исключению. В результате исключение будет обработано *первым* найденным блоком `catch`, который сможет это сделать. Как следствие, в списке директив `catch` наиболее специализированные обработчики следует располагать в начале.

Поскольку поиск директивы `catch` осуществляется в порядке их объявления, при использовании исключений из иерархии наследования блоки `catch` для обработки исключений производного типа следует располагать перед обработчиком для исключения базового типа.

Правила поиска соответствующего исключению блока `catch` значительно жестче, чем правила поиска аргументов, соответствующих типам параметров. Большинство преобразований здесь недопустимо — тип исключения должен точно соответствовать обработчику, допустимо лишь несколько различий.

- Допустимо преобразование из неконстантного типа в константный, т.е. переданный неконстантный объект исключения может быть обработан блоком `catch`, ожидающим ссылку на константный.
- Допустимо преобразование из производного типа в базовый.
- Массив преобразуется в указатель на тип массива; функция преобразуется в соответствующий указатель на тип функции.

Никакие другие преобразования при поиске соответствующего обработчика недопустимы. В частности, невозможны ни стандартные арифметические преобразования, ни преобразования, определенные для классов.



В наборе директив `catch` с типами, связанными наследованием, обработчики для более производных типов следует располагать прежде наименее производных.

Повторная передача исключения

Вполне возможна ситуация, когда один блок кода `catch` (обработчик) не сможет полностью обработать исключение. После некоторых корректирующих действий обработчик может решать, что это исключение следует обработать в функции, которая расположена далее по цепи вызовов. Обработчик может передавать исключение другому, внешнему обработчику, который принадлежит функции, вызвавшей данную. Это называется *повторной передачей исключения* (*rethrow*). Повторную передачу осуществляет оператор `throw`, после которого нет ни имени типа, ни выражения.

```
throw;
```

Пустой оператор `throw` может присутствовать только в обработчике или в функции, вызов которой осуществляется из обработчика (прямо или косвенно). Если пустой оператор `throw` встретится вне обработчика, будет вызвана функция `terminate()`.

Повторная передача не определяет нового исключения; по цепочке передается текущий объект исключения.

Обычно обработчик вполне может изменить содержимое своего параметра. Если после изменения своего параметра обработчик повторно передаст исключение, то эти изменения будут переданы далее, только если параметр обработчика объявлен как ссылка:

```
catch (my_error &eObj) { // спецификатор ссылочного типа
    eObj.status = errCodes::severeErr; // изменение объекта исключения
    throw; // переменная-член status объекта исключения имеет
           // значение severeErr
} catch (other_error eObj) { // спецификатор несылочного типа
    eObj.status = errCodes::badErr; // изменение только локальной копии
    throw; // значение переменной-члена status объекта исключения
           // при повторной передаче не изменилось
}
```

Обработчик для всех исключений

Иногда необходимо обрабатывать все исключения, которые могут произойти, независимо от их типа. Обработка каждого возможного исключения может быть проблематична: иногда неизвестно, исключения каких типов могут быть переданы. Даже когда все возможные типы известны, предоставление отдельной директивы `catch` для каждого возможного исключения может оказаться весьма утомительным. Для обработки всех исключений в объявлении исключения используется многоточие. Такие обработчики, называемые *обработчиками для всех исключений* (*catch-all*), имеют форму `catch(...)`. Такая директива соответствует исключениям любого типа.

Обработчик `catch(...)` зачастую используется в комбинации с выражением повторной передачи. Обработчик осуществляет все локальные действия, а затем повторно передает исключение:

```
void manip() {
    try {
        // действия, приводящие к передаче исключения
    }
    catch (...) {
        // действия по частичной обработке исключения
        throw;
    }
}
```

Директива `catch(...)` применяется самостоятельно или в составе нескольких директив `catch`.



Если директива `catch(...)` используется в комбинации с другими, она должна располагаться последней. Любой обработчик, следующий за обработчиком для всех исключений, никогда не будет выполнен.

Упражнения раздела 18.1.2

Упражнение 18.4. Заглянув вперед в иерархию наследования на рис. 18.1 (стр. 978), объясните, что неправильно в следующем блоке `try`. Исправьте его:

```
try {
    // использовать стандартную библиотеку C++
} catch(exception) {
    // ...
    } catch(const runtime_error &re) {
// ...
} catch(overflow_error eobj) { /* ... */ }
```

Упражнение 18.5. Измените следующую функцию `main()` так, чтобы обрабатывались исключения любых типов, представленных на рис. 18.1 (стр. 978):

```
int main() {
    // использовать стандартную библиотеку C++
}
```

Обработчики должны выводить сообщения об ошибках, связанных с исключением, прежде, чем вызывать функцию `abort()` (определенную в заголовке `cstdlib`) для завершения функции `main()`.

Упражнение 18.6. С учетом следующих типов исключений и директивы `catch` напишите выражение `throw`, создающее объект исключения, который может быть обработан каждым блоком `catch`:

```
(a) class exceptionType { };
    catch(exceptionType *pet) { }
(b) catch(...) { }
(c) typedef int EXCPTYPE;
    catch(EXCPTYPE) { }
```

18.1.3. Блок `try` функции и конструкторы

В принципе исключения могут произойти в любой точке программы. В частности, исключение может произойти в процессе инициализации в конструкторе. Инициализация в конструкторе выполняется прежде, чем его тело. Блок `catch` в теле конструктора не может обработать исключение, которое было передано при инициализации, поскольку блок `try` в теле конструктора еще не был задействован в момент передачи исключения.

Для обработки исключения, переданного при инициализации, конструктор следует оформить как *блок try функции* (function try block). Блок `try функции` позволяет ассоциировать группу директив `catch` с фазой инициализации конструктора (или фазой удаления деструктора), а равно с телом конструктора (или деструктора). В качестве примера заключим конструктор `Blob()` (см. раздел 16.1.2, стр. 834) в блок `try функции`:

```
template <typename T>
Blob<T>::Blob(std::initializer_list<T> il) try {
```

```

        data(std::make_shared<std::vector<T>>(il)) {
            /* пустое тело */
    } catch(const std::bad_alloc &e) { handle_out_of_memory(e); }

```

Обратите внимание на ключевое слово `try`, предшествующее двоеточию, начинающему список инициализации конструктора, и фигурную скобку, формирующую (в данном случае пустое) тело конструктора. Обработчик, связанный с этим блоком `try`, применяется для обработки исключения, переданного либо из списка инициализации, либо из тела конструктора.

Следует заметить, что исключение может произойти при инициализации параметров конструктора. Такие исключения *не являются* частью блока `try` функции. Блок `try` функции обрабатывает только те исключения, которые происходят, когда конструктор начнет выполняться. Как и при любом другом вызове функции, если исключение происходит во время инициализации параметра, оно является частью вызывающего выражения и обрабатывается в контексте вызывающей стороны.



Единственный способ для конструктора обработать исключение из списка инициализации заключается в оформлении конструктора как блока `try` функции.

Упражнения раздела 18.1.3

Упражнение 18.7. Определите классы `Blob` и `BlobPtr` из главы 16 так, чтобы для их конструкторов использовались блоки `try` функции.

18.1.4. Спецификатор исключения `noexcept`

И для пользователей, и для компилятора может быть полезно знать, что функция не будет передавать исключения. Это упрощает написание кода, вызывающего эту функцию. Кроме того, если компилятор знает, что никаких исключений не будет, он может (иногда) оптимизировать код, что недоступно при возможности передачи.



По новому стандарту функция может пообещать не передавать исключения при помощи *спецификации* `noexcept`. Ключевое слово `noexcept` после списка параметров функции означает, что функция не будет передавать исключений:

```

void rescoup(int) noexcept; // не будет передавать исключений
void alloc(int);           // может передавать исключения

```

Эти объявления заявляют, что функция `rescoup()` не будет передавать исключений, а функция `alloc()` могла бы. Считается, что к функции `rescoup()` применена *спецификация запрета передачи исключения* (`nonthrowing specification`).

Спецификатор `noexcept` должен присутствовать во всех объявлениях и в соответствующем определении функции или ни в одном из них. Спецификатор предшествует замыкающему типу (см. раздел 6.3.3, стр. 301). Спецификатор `noexcept` можно определить также в объявлении и определении указателя на функцию. Он неприменим к псевдониму типа или определению типа (`typedef`). В функции-члене спецификатор `noexcept` следует за квалификатором `const` или квалификатором ссылки, но предшествует квалификаторам `final`, `override` и `= 0` у виртуальной функции.

Нарушение спецификации исключения

Важно понимать, что компилятор не проверяет спецификацию `noexcept` во время компиляции. Фактически компилятору не разрешено отклонять функцию со спецификатором `noexcept` просто потому, что она содержит оператор `throw` или вызывает функцию, которая может передавать исключение (однако хорошие компиляторы предупреждают о таких случаях):

```
// эта функция компилируется, хоть она и нарушает свою спецификацию
// исключения
void f() noexcept           // обещание не передавать исключений
{
    throw exception(); // нарушает спецификацию исключения
}
```

В результате вполне вероятно, что функция, обещавшая не передавать исключений, фактически передаст его. Если такая функция передаст исключение, для соблюдения обещания во время выполнения вызывается функция `terminate()`. Результат прокрутки стека непредсказуем. Таким образом, спецификатор `noexcept` следует использовать в двух случаях: если есть уверенность, что функция не будет передавать исключений, или если совершенно неизвестно, как справиться с ошибкой.

Спецификация запрета передачи исключения фактически обещает *вызывающей стороне* такой функции, что ей не придется иметь дела с исключениями. Функция либо не передаст исключения, либо вся программа закончит работу; в любом случае вызывающей стороне не нести ответственность за исключения.



ВНИМАНИЕ

Во время компиляции компилятор может вообще не проверять спецификации исключения.

СОВМЕСТИМОСТЬ С ПРЕЖНЕЙ ВЕРСИЕЙ. СПЕЦИФИКАЦИИ ИСКЛЮЧЕНИЯ

У прежних версий языка C++ была более сложная схема спецификаций исключения, позволяющая определять типы исключений, которые могла бы передавать функция. Функция может определить ключевое слово `throw`, сопровождаемое заключенным в скобки списком типов, которые могла бы

передать функция. Спецификатор `throw` располагается в том же месте, где и спецификатор `noexcept` в текущем языке.

Этот подход никогда широко не использовался и не рекомендован в текущем стандарте. Хотя один случай использования более сложной старой схемы распространен довольно широко. Функция, обозначенная как `throw()`, обещает не передавать никаких исключений:

```
void rescoup(int) noexcept; // rescoup() не передает ничего
void rescoup(int) throw(); // эквивалентное объявление
```

Эти объявления функции `rescoup()` эквивалентны. Оба указывают, что функция `rescoup()` не будет передавать исключений.

Аргументы спецификации `noexcept`

Спецификатор `noexcept` получает необязательный аргумент, тип которого должен быть преобразуем в тип `bool`: если аргументом будет `true`, то функция не будет передавать исключений; если `false` — то может:

```
void rescoup(int) noexcept(true); // не будет передавать исключений
void alloc(int) noexcept(false); // может передавать исключения
```

Оператор `noexcept`

**C++
11** Аргументы спецификатора `noexcept` зачастую создаются с использованием оператора `noexcept`. Оператор `noexcept` — унарный, возвращающий константное логическое выражение `r`-значения, означающее способность данного выражения передавать исключения. Подобно оператору `sizeof` (см. раздел 4.9, стр. 215), оператор `noexcept` не вычисляет свой операнд.

Например, следующее выражение возвращает значение `true`:

```
noexcept(rescoup(i)) // true, если вызов функции rescoup() не может
                     // передать исключение, и false в противном случае
```

поскольку функция `rescoup()` объявлена со спецификатором `noexcept`. В более общем виде выражение `noexcept(e)` возвращает значение `true`, если у всех вызванных `e` функций нет спецификаций передачи и сама `e` не содержит операторов `throw`. В противном случае выражение `noexcept(e)` возвращает значение `false`.

Оператор `noexcept` можно использовать для формирования спецификатора исключения следующим образом:

```
void f() noexcept(noexcept(g())); // f() имеет тот же спецификатор
                                // исключения, что и g()
```

Если функция `g()` обещает не передавать исключений, то `f()` также не будет. Если `g()` не имеет спецификатора исключения или имеет спецификатор, позволяющий передачу исключений, то функция `f()` также может передавать их.



Ключевое слово `noexcept` имеет два значения: это спецификатор исключения, когда оно следует за списком параметров функции, и оператор, который зачастую используется как логический аргумент для спецификатора исключения `noexcept`.

Спецификации исключения и указатели, виртуальные функции, функции управления копированием

Хотя спецификатор `noexcept` не является частью типа функции, наличие у функции спецификатора исключения влияет на ее использование.

Указатель на функцию и функция, на которую указывает этот указатель, должны иметь одинаковые спецификации. Таким образом, если объявлен указатель со спецификатором запрета передачи исключения, то использовать этот указатель можно только для указания на функции с подобным спецификатором. Указатель на функцию, способную передавать исключение, определенный явно или неявно, может указывать на любую функцию, даже если она обещает не передавать исключения:

```
// resoup() и pfl() обещают не передавать исключений
void (*pfl)(int) noexcept = resoup;
// ok: resoup() не будет передавать исключений; и не имеет значения,
// что pf2() может
void (*pf2)(int) = resoup;
pfl = alloc; // ошибка: alloc() может передать исключение, но pfl()
              // обещала, что не будет
pf2 = alloc; // ok: pf2() и alloc() могли бы передать исключение
```

Если виртуальная функция обещает не передавать исключений, унаследованные виртуальные функции также должны обещать не передавать исключений. С другой стороны, если базовая функция позволяет передачу исключения, то производным функциям стоит быть ограниченным строже и обещать не передавать их:

```
class Base {
public:
    virtual double f1(double) noexcept; // не передает исключения
    virtual int f2() noexcept(false);    // может передавать
    virtual void f3();                   // может передавать
};
class Derived : public Base {
public:
    double f1(double); // ошибка: Base::f1() обещает не передавать
    int f2() noexcept(false); // ok: та же спецификация, как у Base::f2()
    void f3() noexcept;       // ok: Derived::f3() ограничена строже
};
```

Когда компилятор синтезирует функции-члены управления копированием, он создает для них спецификацию исключения. Если все соответствующие функции-члены всех базовых классов обещают не передавать исключений, то

синтезируемые функции-члены также будут `noexcept`. Если какая-нибудь функция, вызванная синтезируемым членом, может передать исключение, то этот синтезируемый член помечается как `noexcept(false)`. Кроме того, если разработчик не предоставил спецификацию исключения для деструктора, который он определяет, компилятор синтезирует ее сам. Компилятор создает ту же спецификацию, которую он создал бы, будь то синтезируемый деструктор для этого класса.

Упражнения раздела 18.1.4

Упражнение 18.8. Пересмотрите написанные классы и добавьте соответствующие спецификации исключения к их конструкторам и деструкторам. Если вы полагаете, что некоторые из ваших деструкторов могли бы передавать исключения, изменить код так, чтобы это было невозможно.

18.1.5. Иерархии классов исключений

Классы исключений (см. раздел 5.6.3, стр. 262) стандартной библиотеки формируют иерархию наследования (см. главу 15), представленную на рис. 18.1.

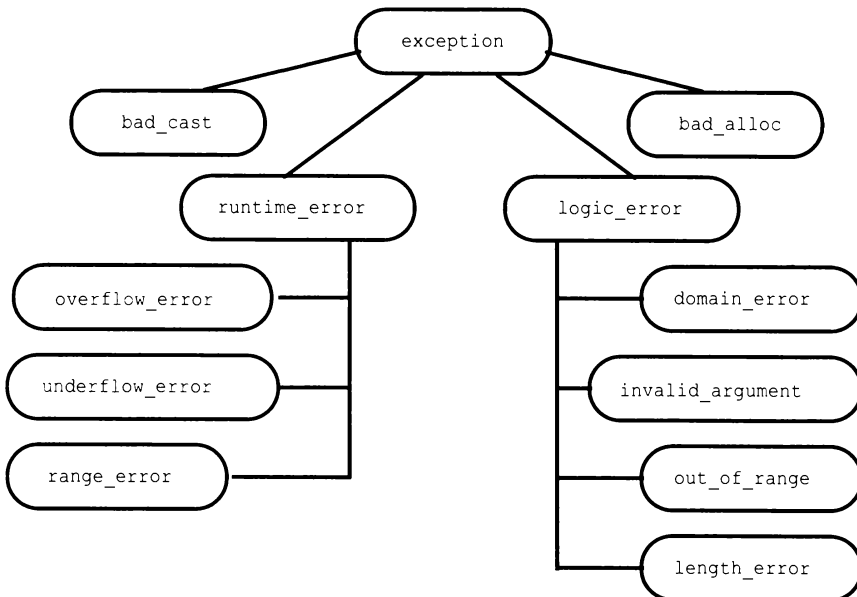


Рис. 18.1. Иерархия классов исключений стандартной библиотеки

Единственными функциями, определенными типом `exception`, являются конструктор копий, оператор присвоения копий, виртуальный деструктор и виртуальная функция-член `what()`. Она возвращает указатель типа `const`

char* на символьный массив с нулевым символом в конце и, как гарантирует, не передает никаких исключений.

Классы исключений `exception`, `bad_cast` и `bad_alloc` определяют также стандартный конструктор. Классы `runtime_error` и `logic_error` не имеют стандартного конструктора, но имеют конструкторы, получающие символьную строку в стиле C или аргумент библиотечного типа `string`. Эти аргументы предназначены для дополнительной информации об ошибке. Функция `what()` этих классов возвращает сообщение, использованное для инициализации объекта исключения. Поскольку функция `what()` виртуальная, при обработке ссылки на базовый тип вызов функции `what()` выполнит ту версию, которая соответствует динамическому типу объекта исключения.

Классы исключения для приложения книжного магазина

В приложениях иерархию исключений зачастую дополняют, определяя классы, производные от класса `exception` (или другого библиотечного класса, производного от него). Такие классы представляют исключения, специфические для данного приложения.

Если бы предстояло создать реальное приложение книжного магазина, его классы были бы гораздо сложнее, чем в примерах этой книги. Одной из причин усложнения является обработка исключений. Фактически пришлось бы создать собственную иерархию исключений, отражающую вероятные проблемы, специфические для данного приложения. В этом проекте могли бы понадобиться следующие классы:

```
// гипотетический класс исключения для приложения книжного магазина
class out_of_stock: public std::runtime_error {
public:
    explicit out_of_stock(const std::string &s):
        std::runtime_error(s) { }
};
class isbn_mismatch: public std::logic_error {
public:
    explicit isbn_mismatch(const std::string &s):
        std::logic_error(s) { }
    isbn_mismatch(const std::string &s,
        const std::string &lhs, const std::string &rhs):
        std::logic_error(s, left(lhs), right(rhs)) { }
    const std::string left, right;
};
```

Здесь специфические для приложения классы исключения определены как производные от стандартного класса исключения. Любую иерархию классов, включая иерархию исключений, можно рассматривать как слоистую структуру. По мере углубления иерархии каждый слой становится более специализированным. Например, первым и наиболее общим слоем иерархии является

класс `exception`. При получении объекта этого типа будет известно только то, что в приложении произошла какая-то ошибка.

Второй слой специализирует исключение на две обширные категории: ошибки времени выполнения и логические ошибки. Ошибки времени выполнения могут быть обнаружены только при запуске программы. Логические ошибки, в принципе, могут быть обнаружены в приложении.

Классы исключений книжного магазина представляют даже более специализированный слой. Класс `out_of_stock` представляет проблему времени выполнения, специфическую для данного приложения. Он используется для оповещения о нарушении порядка выполнения. Класс исключения `isbn_mismatch` представляет собой более специализированную форму класса `logic_error`. В принципе программа может обнаружить несоответствие ISBN, вызвав функцию `isbn()`.

Использование собственных типов исключений

Собственные классы исключений применяются точно так же, как и классы стандартной библиотеки. Одна часть программы передает объект одного из этих классов, а другая получает и обрабатывает его, устраняя проблему. Например, для перегруженного оператора суммы класса `Sales_item` можно создать класс исключения `isbn_mismatch`, передаваемого в случае обнаружения ошибки несовпадения ISBN.

```
// передает исключение, если isbn объектов не совпадают
Sales_data&
Sales_data::operator+=(const Sales_data& rhs)
{
    if (isbn() != rhs.isbn())
        throw isbn_mismatch("wrong isbn", isbn(), rhs.isbn());
    units_sold += rhs.units_sold;
    revenue += rhs.revenue;
    return *this;
}
```

Обнаружив эту ошибку, использующий оператор `+=` код сможет передать соответствующее сообщение об ошибке и продолжить работу.

```
// применение исключения в приложении книжного магазина
Sales_data item1, item2, sum;
while (cin >> item1 >> item2) { // прочитать две транзакции
    try {
        sum = item1 + item2; // вычислить их сумму
        // использовать сумму
    } catch (const isbn_mismatch &e) {
        cerr << e.what() << ": left isbn(" << e.left
            << ") right isbn(" << e.right << ")" << endl;
    }
}
```

Упражнения раздела 18.1.5

Упражнение 18.9. Определите описанные в этом разделе классы исключений приложения книжного магазина и перепишите составной оператор присвоения класса `Sales_data` так, чтобы он передавал исключение.

Упражнение 18.10. Напишите программу, использующую оператор суммы класса `Sales_data` для объектов с разными ISBN. Напишите две версии программы: способную обрабатывать исключения и не обрабатывающую их. Сравните поведение программ, чтобы ознакомиться с тем, что происходит при отсутствии обработки исключения.

Упражнение 18.11. Почему так важно, чтобы функция `what()` не передавала исключений?

18.2. Пространства имен

В больших программах обычно используют библиотеки от независимых разработчиков. В таких библиотеках обычно определено множество глобальных имен классов, функций и шаблонов. Когда приложение использует библиотеки от многих разных поставщиков, некоторые из этих имен почти неизбежно совпадут. Библиотеки, помещающие имена в глобальное пространство имен, вызывают *загромождение пространства имен* (*namespace pollution*).

Традиционно программисты избегают загромождения пространства имен, используя для глобальных сущностей очень длинные имена, зачастую содержащие префикс, означающий библиотеку, в которой определено имя:

```
class cplusplus_primer_Query { ... };  
string cplusplus_primer_make_plural(size_t, string&);
```

Это решение далеко от идеала: программистам неудобно писать и читать программы, использующие длинные имена.

Пространства имен (*namespace*) предоставляют намного более контролируемый механизм предотвращения конфликтов имени. Пространства имен разделяют глобальное пространство имен. Пространство имен — это область видимости. При определении имен библиотеки в пространстве имен, авторы (и пользователи) библиотеки могут избежать ограничений, присущих глобальным именам.

18.2.1. Определение пространств имен

Определение пространства имен начинается с ключевого слова `namespace`, сопровождаемого именем пространства имен. После имени пространства имен следуют заключенные в фигурные скобки объявления и определения. В пространство имен может быть помещено любое объявление, которое спо-

собно присутствовать в глобальной области видимости, включая классы, переменные (с инициализацией), функции (с их определениями), шаблоны и другие пространства имен.

```
namespace cplusplus_primer {
    class Sales_data { /* ... */};
    Sales_data operator+(const Sales_data&,
                        const Sales_data&);
    class Query { /* ... */ };
    class Query_base { /* ... */};
} // подобно блокам, пространства имен не завершаются точкой с запятой
```

Этот код определяет пространство имен `cplusplus_primer` с четырьмя членами: тремя классами и перегруженным оператором `+`.

Подобно другим именам, имя пространства имен должно быть уникальным в той области видимости, в которой оно определено. Пространства имен могут быть определены в глобальной области видимости или в другом пространстве имен. Они не могут быть определены в функциях или классах.



Область видимости пространства имен не заканчивается точкой с запятой.

Каждое пространство имен является областью видимости

Как и в случае любой области видимости, каждое имя в пространстве имен должно относиться к уникальной сущности в пределах данного пространства имен. Поскольку разные пространства имен вводят разные области видимости, в разных пространствах имен могут быть члены с одинаковым именем.

К именам, определенным в пространстве имен, другие члены данного пространства имен могут обращаться непосредственно, включая области видимости, вложенные в пределах этих членов. Код вне пространства имен должен указывать пространство имен, в котором определено имя:

```
cplusplus_primer::Query q =
    cplusplus_primer::Query("hello");
```

Если другое пространство имен (например, `AddisonWesley`) тоже содержит класс `Query` и этот класс необходимо использовать вместо определенного в пространстве имен `cplusplus_primer`, приведенный выше код придется изменить следующим образом:

```
AddisonWesley::Query q = AddisonWesley::Query("hello");
```

Пространства имен могут быть разобщены

Как упоминалось в разделе 16.5 (стр. 890), в отличие от других областей видимости, пространство имен может быть определено в нескольких частях. Вот определение пространства имен:

```
namespace nsp {
    // объявления
}
```

Этот код определяет новое пространство имен `nsp` или добавляет члены к уже существующему. Если пространство имен `nsp` еще не определено, то создается новое пространство имен с этим именем. В противном случае это определение открывает уже существующее пространство имен и добавляет в него новые объявления.

Тот факт, что определения пространств имен могут быть разобщены, позволяет составить пространство имен из отдельных файлов интерфейса и реализации. Таким образом, пространство имен может быть организовано таким же образом, как и определения собственных классов или функций.

- Члены пространства имен, являющиеся определениями классов, объявлениями функций и объектов, составляющих часть интерфейса класса, могут быть помещены в файлы заголовка. Эти заголовки могут быть подключены в те файлы, которые используют эти члены пространства имен.
- Определения членов пространства имен могут быть помещены в отдельные файлы исходного кода.

Организовав пространство имен таким образом, можно также удовлетворить требование, согласно которому различные сущности, включая не подлежащие встраиванию функции, статические переменные-члены, переменные и т.д., должны быть определены в программе только один раз. Это требование распространяется и на имена, определенные в пространстве имен. Отделив интерфейс и реализацию, можно гарантировать, что имена функций и другие имена будут определены только один раз и именно это объявление будет многократно использоваться впоследствии.



Для представления несвязанных типов в составных пространствах имен следует использовать отдельные файлы.

Определение пространства имен `cplusplus_primer`

Используя эту стратегию для отделения интерфейса от реализации, определим библиотеку `cplusplus_primer` в нескольких отдельных файлах. Объявления класса `Sales_data` и связанных с ним функций поместим в файл заголовка `Sales_data.h`, а таковые для класса `Query` (см. главу 15) — в заголовок `Query.h` и т.д. Соответствующие файлы реализации были бы в таких файлах, как `Sales_data.cc` и `Query.cc`:

```
// ---- Sales_data.h ----
// директивы #includes должны быть перед открытием пространства имен
#include <string>
namespace cplusplus_primer {
```

```

class Sales_data { /* ... */;
Sales_data operator+(const Sales_data&,
                    const Sales_data&);
    // объявления остальных функций интерфейса класса Sales_data
}
// ---- Sales_data.cc ----
// все директивы #includes перед открытием пространства имен
#include "Sales_data.h"
namespace cplusplus_primer {
    // определения членов класса Sales_data и перегруженных операторов
}

```

Использующая эту библиотеку программа включила бы все необходимые заголовки. Имена в этих заголовках определены в пространстве имен `cplusplus_primer`:

```

// ---- user.cc ----
// имена заголовка Sales_data.h находятся в пространстве
// имен cplusplus_primer
#include "Sales_data.h"
int main()
{
    using cplusplus_primer::Sales_data;
    Sales_data trans1, trans2;
    // ...
    return 0;
}

```

Подобная организация программы придает библиотеке свойство модульности, необходимое как разработчикам, так и пользователям. Каждый класс организован в виде двух файлов: интерфейса и реализации. Пользователь одного класса вовсе не должен использовать при компиляции другие классы. Их реализацию можно скрыть от пользователей, разрешив при этом компилировать и компоновать файлы `Sales_data.cc` и `user.cc` в одну программу, причем без опасений по поводу возникновения ошибок во время компиляции или компоновки. Кроме того, разработчики библиотеки могут работать над реализацией каждого класса независимо.

В использующую эту библиотеку программу следует подключить все необходимые заголовки. Имена в этих заголовках определены в пространстве имен `cplusplus_primer`.

Следует заметить, что директивы `#include` обычно не помещают в пространство имен. Если попробовать сделать это, то произойдет попытка определения всех имен в этом заголовке как членов окружающего пространства имен. Например, если бы файл `Sales_data.h` открыл пространство имен `cplusplus_primer` прежде, чем включить заголовок `string`, то в программе была бы ошибка, поскольку это привело бы к попытке определить пространство имен `std` в пространстве имен `cplusplus_primer`.

Определение членов пространства имен

Если объявления находятся в области видимости, то код в пространстве имен может использовать короткую форму имен, определенных в том же (или вложенном) пространстве имен:

```
#include "Sales_data.h"
namespace cplusplus_primer { // повторное открытие cplusplus_primer
// члены, определенные в пространстве имен, могут использовать имена
// без уточнений
    std::istream&
    operator>>(std::istream& in, Sales_data& s) { /* ... */}
}
```

Член пространства имен может быть также определен вне определения пространства имен. Для этого применяется подход, подобный определению членов класса вне его. Объявление пространства имен должно находиться в области видимости, а в определении следует указать пространство имен, которому принадлежит имя.

```
// члены пространства имен, определенные вне его, должны использовать
// полностью квалифицированные имена
cplusplus_primer::Sales_data
cplusplus_primer::operator+(const Sales_data& lhs,
                             const Sales_data& rhs)
{
    Sales_data ret(lhs);
    // ...
}
```

Подобно членам класса, определенным вне самого класса, когда встречается полностью определенное имя, оно находится в пределах пространства имен. В пространстве имен `cplusplus_primer` можно использовать другие имена членов пространства имен без квалификации. Таким образом, хотя класс `Sales_data` является членом пространства имен `cplusplus_primer`, для определения параметров его функций можно использовать его имя без квалификации.

Хотя член класса пространства имен может быть определен вне его определения, такие определения должны присутствовать в окружающем пространстве имен. Таким образом, оператор `operator+` класса `Sales_data` можно определить в пространстве имен `cplusplus_primer` или в глобальной области видимости. Но он не может быть определен в несвязанном пространстве имен.

Специализация шаблона

Специализация шаблона должна быть определена в том же пространстве имен, которое содержит первоначальный шаблон (см. разделе 16.5, стр. 890). Подобно любым другим именам пространства имен, пока специализация объявлена в пространстве имен, ее можно определить вне пространства имен:

```
// специализацию нужно объявить как член пространства std
namespace std {
    template <> struct hash<Sales_data>;
}
// добавив объявление для специализации к пространству std,
// специализацию можно определить вне пространства имен std
template <> struct std::hash<Sales_data>
{
    size_t operator()(const Sales_data& s) const
    { return hash<string>()(s.bookNo) ^
        hash<unsigned>()(s.units_sold) ^
        hash<double>()(s.revenue); }
    // другие члены как прежде
};
```

Глобальное пространство имен

Имена, определенные в глобальной области видимости (т.е. имена, объявленные вне любого класса, функции или пространства имен), определяются в *глобальном пространстве имен* (global namespace). Глобальное пространство имен неявно объявляется и существует в каждом приложении. Каждый файл, который определяет сущность в глобальной области видимости (неявно), добавляет ее имя к глобальному пространству имен.

Для обращения к членам глобального пространства имен применяется *оператор области видимости* (оператор `::`) (scope operator). Поскольку глобальное пространство имен неявно, у него нет имени.

Форма записи при обращении к члену глобального пространства имен имеет следующий вид.

```
::член_имя
```

Вложенные пространства имен

Вложенное пространство имен (nested namespace) — это пространство имен, определенное в другом пространстве имен:

```
namespace cplusplus_primer {
    // первое вложенное пространство имен: определение части
    // библиотеки Query
    namespace QueryLib {
        class Query { /* ... */ };
        Query operator&(const Query&, const Query&);
        // ...
    }

    // второе вложенное пространство имен: определение части
    // библиотеки Sales_data
    namespace Bookstore {
        class Quote { /* ... */ };
        class Disc_quote : public Quote { /* ... */ };
        // ...
    }
}
```

Вложенное пространство имен — это вложенная область видимости, ее область видимости вкладывается в пределы содержащего ее пространства имен. Имена вложенных пространств имен подчиняются обычным правилам: имена, объявленные во внутреннем пространстве имен, скрывают объявления того же имени во внешнем пространстве. Имена, определенные во вложенном пространстве имен, являются локальными для внутреннего пространства имен. Код во внешних частях окружающего пространства имен может обратиться к имени во вложенном пространстве имен только через его квалифицированное имя. Например, имя класса `QueryLib`, объявленного во вложенном пространстве имен, выглядит следующим образом:

```
cplusplus_primer::QueryLib::Query
```

Встраиваемые пространства имен

C++ 11 Новый стандарт ввел новый вид вложенного пространства имен — *встраиваемое пространство имен* (*inline namespace*). В отличие от обычных вложенных пространств имен, имена из встраиваемого пространства имен применяются так, как будто они являются непосредственными членами окружающего пространства имен. Таким образом, нет необходимости в квалификации имен из встраиваемого пространства имен. Для доступа к ним достаточно использовать имя окружающего пространства имен.

Для определения встраиваемого пространства имен ключевое слово `namespace` предваряется ключевым словом `inline`:

```
inline namespace FifthEd {
    // пространство имен для кода Primer Fifth Edition
}
namespace FifthEd { // неявно встраиваемая
    class Query_base { /* ... */};
    // другие объявления, связанные с классом Query
}
```

Это ключевое слово должно присутствовать в первом определении пространства имен. Если пространство имен вновь открывается позже, ключевое слово `inline` не обязательно, но может быть повторено.

Встраиваемые пространства имен зачастую используются при изменении кода от одного выпуска приложения к следующему. Например, весь код текущего издания *Вводного курса* можно поместить во встраиваемое пространство имен. Код предыдущих версий был бы в обычных, а не встраиваемых пространствах имен:

```
namespace FourthEd {
    class Item_base { /* ... */};
    class Query_base { /* ... */};
    // другой код из Fourth Edition
}
```

Общее пространство имен `cplusplus_primer` включило бы определения обоих пространств имен. Например, с учетом того, что каждое пространство имен было определено в заголовке с соответствующим именем, пространство имен `cplusplus_primer` можно определить следующим образом:

```
namespace cplusplus_primer {  
#include "FifthEd.h"  
#include "FourthEd.h"  
}
```

Поскольку пространство имен `FifthEd` встраиваемое, код обращающийся к имени из пространства имен `cplusplus_primer::`, получит версию из этого пространства имен. Если понадобится код прежнего издания, к нему можно обратиться как к любому другому вложенному пространству имен, указав все имена окружающих пространств имен, например: `cplusplus_primer::FourthEd::Query_base`.

Безымянные пространства имен

У *безымянного пространства имен* (`unnamed namespace`) сразу за ключевым словом `namespace` следует блок объявлений, разграниченных фигурными скобками. У переменных, определенных в безымянном пространстве имен, статическая продолжительность существования: они создаются перед их первым использованием и удаляются по завершении программы.

Безымянное пространство имен может быть разобщено в пределах данного файла, но не охватывающих файлов. У каждого файла есть собственное безымянное пространство имен. Если два файла содержат безымянные пространства имен, эти пространства имен не связаны. Оба безымянных пространства имен могут определить одинаковое имя, и эти определения будут относиться к разным сущностям. Если заголовок определяет безымянное пространство имен, то имена в этом пространстве определяют сущности, локальные для каждого файла, включенного в заголовок.



В отличие от других пространств имен, безымянное пространство является локальным для конкретного файла и никогда не охватывает несколько файлов.

Имена, определенные в безымянном пространстве имен, используются непосредственно; в конце концов, для их квалификации нет никакого имени пространства имен. Для обращения к членам безымянных пространств имен невозможно использовать оператор области видимости.

Имена, определенные в безымянном пространстве имен, находятся в той же области видимости, что и область видимости, в которой определено пространство имен. Если безымянное пространство имен определяется в наиболее удаленной области видимости файла, то имена в безымянном простран-

стве имен должны отличаться от имен, определенных в глобальной области видимости:

```
int i; // глобальное объявление для i
namespace {
    int i;
}
// неоднозначность: определено глобально и в не вложенном, безымянном
// пространстве имен
i = 10;
```

Во всем остальном члены безымянного пространства имен являются обычными сущностями программы. Безымянное пространство имен, как и любое другое пространство имен, может быть вложено в другое пространство имен. Если безымянное пространство имен вкладывается, то к содержащимся в нем именам обращаются обычным способом, используя имена окружающего пространства имен:

```
namespace local {
    namespace {
        int i;
    }
}
// ок: i определено во вложенном безымянном пространстве имен отдельно от
// глобального i
local::i = 42;
```

БЕЗЫМЯННЫЕ ПРОСТРАНСТВА ИМЕН ВМЕСТО СТАТИЧЕСКИХ ФАЙЛОВЫХ ОБЪЕКТОВ

До введения пространств имен в стандарт C++, чтобы сделать имена локальными для файла, их приходилось объявлять статическими (*static*). Применение *статических файловых объектов* (*file static*) унаследовано от языка C. В языке C объявленный статическим глобальный объект был невидим вне того файла, в котором он объявлен.



В соответствии со стандартом C++ применение объявлений статических файловых объектов не рекомендуется. Вместо них используются безымянные пространства имен.

Упражнения раздела 18.2.1

Упражнение 18.12. Организуйте программы, написанные в упражнениях каждой из глав, в их собственные пространства имен. Таким образом, пространство имен `chapter15` содержало бы код для программы запросов, а `chapter10` — код приложения `TextQuery`. Используя эту структуру, откомпилируйте примеры кода приложения `Query`.

Упражнение 18.13. Когда используются безымянные пространства имен?

Упражнение 18.14. Предположим, имеется следующее объявление оператора `operator*`, являющегося членом вложенного пространства имен `mathLib::MatrixLib`:

```
namespace mathLib {
    namespace MatrixLib {
        class matrix { /* ... */ };
        matrix operator*
            (const matrix &, const matrix &);
        // ...
    }
}
```

Как определить этот оператор в глобальной области видимости?

18.2.2. Использование членов пространства имен

Обращение к члену пространства имен в формате *имя_пространства_имен::имя_члена* является чересчур громоздким, особенно когда имя пространства имен слишком длинное. К счастью, существуют способы, которые облегчают использование имен членов пространства имен. Один из этих способов, объявление `using` (см. раздел 3.1, стр. 124), уже использовался в программах, приведенных выше. Другие способы, псевдонимы пространств имен и директивы `using` будут описаны в этом разделе.

Псевдонимы пространства имен

Псевдоним пространства имен (`namespace alias`) применяется в качестве короткого синонима имени пространства имен. Например, длинное имя пространства имен может иметь следующий вид:

```
namespace cplusplus_primer { /* ... */ };
```

Ему может быть назначен более короткий синоним следующим образом:

```
namespace primer = cplusplus_primer;
```

Объявление псевдонима пространства имен начинается с ключевого слова `namespace`, за которым следует имя псевдонима пространства имен (короткое), сопровождаемое знаком `=`, первоначальное имя пространства имен и точка с запятой. Если имя первоначального пространства имен еще не было определено как пространство имен, произойдет ошибка.

Псевдоним пространства имен может быть также применен к вложенному пространству имен:

```
namespace Qlib = cplusplus_primer::QueryLib;
Qlib::Query q;
```



Пространство имен может иметь множество синонимов или псевдонимов. Все псевдонимы и первоначальное имя пространства имен равнозначны в применении.

Объявления `using` (напоминание)

Имена, представленные в объявлении `using`, подчиняются обычным правилам области видимости. Имя видимо от точки объявления `using` и до конца области видимости, в которой оно объявлено. Сущности внутренней области видимости скрывают одноименные сущности внешней. Короткие имена могут использоваться только в той области видимости, в которой они объявлены, а также в областях видимости, вложенных в нее. По завершении области видимости следует использовать полные имена.

Объявление `using` может присутствовать в глобальной и локальной области видимости, а также в области видимости пространства имен или класса. Объявление `using` в области видимости класса ограничено именами, определенными в базовом классе определяемого класса (см. раздел 15.5, стр. 776).

Директива `using`

Подобно объявлению `using`, *директива* `using` (`using directive`) позволяет использовать не квалифицированную форму имен. Однако, в отличие от объявления `using`, здесь не сохраняется контроль над видимостью имен, поскольку все они видимы.

Директива `using` начинается с ключевого слова `using`, за которым следует ключевое слово `namespace`, сопровождаемое именем пространства имен. Если имя пространства не было определено ранее, произойдет ошибка. Директива `using` может присутствовать в глобальной, локальной области видимости или в пространстве имен. Она не может присутствовать в области видимости класса.



Предоставление директив `using` для таких пространств имен, как `std`, которые приложение не контролирует, возвращает все проблемы конфликта имени, присущие использованию нескольких библиотек.

Директива `using` и область видимости

Область видимости имен, указанных директивой `using`, гораздо сложнее, чем в случае объявления `using`. Объявление `using` помещает имя непосредственно в ту же область видимости, в которой находится само объявление `using`. Объявление `using` подобно локальному псевдониму для члена пространства имен.

Директива `using` не объявляет локальные псевдонимы для имен членов пространства имен. Вместо этого она поднимает члены пространства имен

в ближайшую область видимости, которая содержит и пространство имен, и саму директиву `using`.

Различие в области видимости между объявлением `using` и директивой `using` проистекает непосредственно из принципа действия этих средств. В случае объявления `using` само имя просто становится доступным в локальной области видимости. Директива `using`, напротив, делает доступным все содержимое пространства имен. Вообще, пространство имен способно включать определения, которые не могут присутствовать в локальной области видимости. Как следствие, директива `using` рассматривается как присутствующая в ближайшей области видимости окружающего пространства имен.

Рассмотрим самый простой случай. Предположим, что в глобальной области видимости определено пространство имен `A` и функция `f()`. Если функция `f()` имеет директиву `using` для пространства имен `A`, функция `f()` будет вести себя так, как будто имена пространства имен `A` присутствовали в глобальной области видимости до определения функции `f()`.

```
// пространство имен A и функция f() определены в глобальной области
// видимости
namespace A {
    int i, j;
}
void f()
{
    using namespace A; // переводит имена из области видимости A в
                       // глобальную область видимости
    cout << i * j << endl; // использует i и j из пространства имен A
    // ...
}
```

Пример директив `using`

Рассмотрим следующий пример:

```
namespace blip {
    int i = 16, j = 15, k = 23;
    // другие объявления
}
int j = 0; // ok: j в пространстве имен blip скрыта
void manip()
{
    // директива using; имена пространства имен blip "добавляются" к
    // глобальной области видимости
    using namespace blip; // конфликт между ::j и blip::j
    // обнаруживается только при использовании j
    ++i;           // присваивает blip::i значение 17
    ++j;           // ошибка неоднозначности: global j или blip::j?
    ++::j;         // ok: присваивает глобальной j значение 1
    ++blip::j;     // ok: присваивает blip::j значение 16
    int k = 97;    // локальная k скрывает blip::k
    ++k;           // присваивает локальной k значение 98
}
```


Директива `using` в функции `manip()` делает все имена пространства имен `blip` доступными непосредственно. То есть функция `manip()` может обращаться к этим членам, используя краткую форму имен.

Члены пространства имен `blip` выглядят так, как будто они были определены в одной области видимости. Если пространство имен `blip` определено в глобальной области видимости, его члены будут выглядеть так, как будто они объявлены в глобальной области видимости.

Когда пространство имен вводится в окружающую область видимости, имена в пространстве имен вполне могут вступить в конфликт с другими именами, определенными (включенными) в той же области видимости. Например, в функции `manip()` член `j` пространства имен `blip` вступает в конфликт с глобальным объектом `j`. Такие конфликты разрешимы, но для использования имени следует явно указать, какая версия имеется в виду. Любое использование имени `j` в пределах функции `manip()` ведет к неоднозначности.

Чтобы использовать такое имя, как `j`, следует применить оператор области видимости, позволяющий указать требуемое имя. Для указания переменной `j`, определенной в глобальной области видимости, нужно написать `::j`, а для определенной в пространстве имен `blip` — `blip::j`.

Поскольку имена находятся в разных областях видимости, локальные объявления в пределах функции `manip()` могут скрыть некоторые из имен пространства имен. Локальная переменная `k` скрывает член пространства имен `blip::k`. Обращение к переменной `k` в пределах функции `manip()` вполне однозначно, это обращение к локальной переменной `k`.

Заголовки и объявления `using` или директивы

Заголовок, содержащий директиву или объявление `using` в своей области видимости верхнего уровня, вводит свои имена в каждый файл, который подключает заголовок. Обычно заголовки должны определять только те имена, которые являются частью его интерфейса, но не имена, используемые в его реализации. В результате файлы заголовка не должны содержать директив или объявлений `using`, кроме как в функциях или пространствах имен (см. раздел 3.1, стр. 125).

ВНИМАНИЕ! ИЗБЕГАЙТЕ ДИРЕКТИВ `using`

Директивы `using`, вводящие в область видимости все имена из пространства имен, обманчиво просты в использовании. Единственный оператор делает видимыми имена всех членов пространства имен. Хотя этот подход может показаться простым, он создает немало проблем. Если в приложении использовано много библиотек и директива `using` сделает видимыми имена, определенные в них, то вновь возникнет проблема загромождения глобального пространства имен.

Кроме того, не исключено, что при выходе новой версии библиотеки вполне работоспособная в прошлом программа перестанет компилироваться. Причиной этой проблемы может быть конфликт имен новой версии с именами, которые использовались прежде.

Еще одна вызванная директивой `using` проблема неоднозначности обнаруживается только в момент применения. Столь позднее обнаружение означает, что конфликты могут возникать значительно позже применения определенной библиотеки. То есть при использовании в программе новой библиотеки могут возникнуть необнаруженные ранее конфликты.

Поэтому лучше не полагаться на директиву `using` и использовать объявление `using` для каждого конкретного имени пространства имен, используемого в программе. Это уменьшит количество имен, вводимых в пространство имен. Кроме того, ошибки неоднозначности, причиной которых является объявление `using`, обнаруживаются в точке объявления, а это существенно упрощает их поиск.



Директивы `using` на самом деле полезны в файлах реализации самого пространства имен.

Упражнения раздела 18.2.2

Упражнение 18.15. Объясните различия между объявлением и директивой `using`.

Упражнение 18.16. Объясните следующий код с учетом того, что объявления `using` для всех членов пространства имен `Exercise` находятся в области, помеченной как *позиция 1*. Что, если вместо этого они располагаются в позиции 2? Теперь ответьте на тот же вопрос, но замените объявления `using` директивой `using` для пространства имен `Exercise`.

```
namespace Exercise {
    int ivar = 0;
    double dvar = 0;
    const int limit = 1000;
}
int ivar = 0;
// позиция 1
void manip() {
    // позиция 2
    double dvar = 3.1416;
    int iobj = limit + 1;
    ++ivar;
    ++::ivar;
}
```

Упражнение 18.17. Напишите код для проверки ответов на предыдущий вопрос.

18.2.3. Классы, пространства имен и области видимости

Поиск имен, используемых в пространстве имен, происходит согласно обычным правилам поиска в языке C++: сначала во внутренней, а затем во внешней области видимости. Имя, используемое в пространстве имен, может быть определено в одном из окружающих пространств имен, включая глобальное пространство имен. Однако учитываются только те имена, которые были объявлены перед точкой использования в блоках, которые все еще открыты.

```
namespace A {
    int i;
    namespace B {
        int i; // скрывает A::i в B
        int j;
        int f1()
        {
            int j;    // j локальна для f1() и скрывает A::B::j
            return i; // возвращает B::i
        }
    } // пространство имен B закрыто, и его имена больше не видимы
    int f2() {
        return j; // ошибка: j не определена
    }
    int j = i;    // инициализируется значением A::i
}
```

Когда класс расположен в пространстве имен, процесс поиска остается обычным: когда имя используется функцией-членом, его поиск начинается в самой функции, затем в пределах класса (включая базовые классы), а потом в окружающих областях видимости, одной или несколькими из которых могли бы быть пространства имен:

```
namespace A {
    int i;
    int k;
    class C1 {
    public:
        C1(): i(0), j(0) { } // ok: инициализирует C1::i и C1::j
        int f1() { return k; } // возвращает A::k
        int f2() { return h; } // ошибка: h не определена
        int f3();
    private:
        int i; // скрывает A::i в C1
        int j;
    };
    int h = i; // инициализируется значением A::i
}
// член f3() определен вне класса C1 и вне пространства имен A
int A::C1::f3() { return h; } // ok: возвращает A::h
```

За исключением определений функций-членов, расположенных в теле класса (см. раздел 7.4.1, стр. 367), области видимости всегда просматриваются снизу вверх: имя должно быть объявлено прежде его применения. Следовательно, оператор `return` функции `f2()` не будет откомпилирован. Он попытается обратиться к имени `h` из пространства имен `A`, но там оно еще не определено. Если бы это имя `h` было определено в пространстве имен `A` прежде определения класса `C1`, его использование было бы вполне допустимо. Аналогично использование имени `h` в функции `f3()` вполне допустимо, поскольку функция `f3()` определена уже после определения `A::h`.



Порядок просмотра областей видимости при поиске имени определяется по полностью квалифицированному имени функции. Полностью квалифицированное имя указывает в обратном порядке области видимости, в которых происходит поиск.

Спецификаторы `A::C1::f3()` указывают обратный порядок, в котором просматриваются области видимости класса и пространств имен. Первая область видимости — это функция `f3()`. Далее следует область видимости ее класса `C1`. Область видимости пространства имен `A` просматривается в последнюю очередь, перед переходом к области видимости, содержащей определение функции `f3()`.



Зависимый от аргумента поиск и параметры типа класса

Рассмотрим простую программу:

```
std::string s;
std::cin >> s;
```

Как известно, этот вызов эквивалентен следующему (см. раздел 14.1, стр. 701):

```
operator>>(std::cin, s);
```

Функция `operator>>` определена библиотекой `string`, которая в свою очередь определяется в пространстве имен `std`. Но все же оператор `>>` можно вызвать без спецификатора `std::` и без объявления `using`.

Непосредственно обратиться к оператору вывода можно потому, что есть важное исключение из правила сокрытия имен, определенных в пространстве имен. Когда объект класса передается функции, компилятор ищет пространство имен, в котором определяется класс аргумента *в дополнение* к обычному поиску области видимости. Это исключение применимо также к вызовам с передачей указателей или ссылок на тип класса.

В этом примере, когда компилятор встречает “вызов” оператора `operator>>`, он ищет соответствующую функцию в текущей области видимости, включая области видимости, окружающие оператор вывода. Кроме того,

поскольку выражение вывода имеет параметры типа класса, компилятор ищет также в пространствах имен, в которых определяются типы `cin` и `s`. Таким образом, для этого вызова компилятор просмотрит пространство имен `std`, определяющее типы `istream` и `string`. При поиске в пространстве имен `std` компилятор находит функцию вывода класса `string`.

Это исключение из правил поиска позволяет функции, не являющейся членом класса, быть концептуально частью интерфейса к классу и использоваться без отдельного объявления `using`. Без этого исключения из правил поиска для оператора вывода всегда пришлось бы предоставлять соответствующее объявление `using`:

```
using std::operator>>; // чтобы позволить cin >> s
```

Либо пришлось бы использовать форму записи вызова функции, включающую спецификатор пространства имен:

```
std::operator>>(std::cin, s); // ok: явное использование std::>>
```

Не было бы никакого способа использовать синтаксис оператора. Любое из этих объявлений выглядело неуклюже и существенно затруднило бы использование библиотеки ввода-вывода.

Поиск и функции `std::move()` и `std::forward()`

Многим, возможно, даже большинству программистов C++ никогда не понадобится зависимый от аргумента поиск. Обычно, если приложение определяет имя, уже определенное в библиотеке, истинно одно из двух: либо обычные правила перегрузки определяют, относится ли данный конкретный вызов к библиотечной версии функции, или к версии приложения, или приложение никогда не сможет использовать библиотечную функцию.

Теперь рассмотрите библиотечные функции `move()` и `forward()`. Обе являются шаблонами функций, и библиотека определяет их версии с одним параметром функции в виде ссылки на *r*-значение. Как уже упоминалось, параметру ссылки на *r*-значение в шаблоне функции может соответствовать любой тип (см. раздел 16.2.6, стр. 867). Если приложение определяет функцию по имени `move()`, получающую один параметр, то (вне зависимости от типа параметра) версия функции `move()` из приложения вступит в конфликт с библиотечной версией. Это справедливо и для функции `forward()`.

В результате конфликты имен для функций `move()` (и `forward()`) более вероятны, чем для других библиотечных функций. Кроме того, поскольку функции `move()` и `forward()` осуществляют весьма специфические для типа манипуляции, вероятность того, что в приложении специально необходимо переопределить поведение этих функций, довольно мала.

Тот факт, что конфликты имен с этими функциями более вероятны (и менее вероятно, что намеренными), объясняет, почему их имена всегда следует использовать полностью квалифицированными (см. раздел 12.1.5, стр. 598). Форма записи `std::move()`, а не просто `move()` гарантирует применение версии из стандартной библиотеки.



Дружественные объявления и зависимый от аргумента поиск

Напомним, что на момент, когда класс объявляет функцию дружественной (см. раздел 7.2.1, стр. 351), объявление функции необязательно должно быть видимым. Если объявление функции еще не видимо, результатом объявления ее дружественной окажется помещение объявления данной функции или класса в окружающую область видимости. Комбинация этого правила и зависимого от аргумента поиска может привести к неожиданным результатам:

```
namespace A {
    class C {
        // два друга; ничего не объявлено кроме дружественных отношений
        // эти функции неявно являются членами пространства имен A
        friend void f2(); // не будет найдено, если не объявлено иное
        friend void f(const C&); // найдено зависимым от аргумента
                                // поиском
    };
}
```

Здесь функции `f()` и `f2()` являются членами пространства имен `A`. Зависимый от аргумента поиск позволяет вызвать функцию `f()`, даже если для нее нет никакого дополнительного объявления:

```
int main()
{
    A::C cobj;
    f(cobj); // ok: находит A::f() по объявлению дружественным в A::C
    f2();    // ошибка: A::f2() не объявлена
}
```

Поскольку функция `f()` получает аргумент типа класса и неявно объявляется в том же пространстве имен, что и `C`, при вызове она будет найдена. Так как у функции `f2()` никакого параметра нет, она не будет найдена.

Упражнения раздела 18.2.3

Упражнение 18.18. С учетом следующего типичного определения функции `swap()` в разделе 13.3 (стр. 655) определите, какая ее версия используется, если `mem1` имеет тип `string`. Что, если `mem1` имеет тип `int`? Объясните, как будет проходить поиск имен в обоих случаях.

```
void swap(T v1, T v2)
{
    using std::swap;
    swap(v1.mem1, v2.mem1);
    // обмен остальных членов типа T
}
```

Упражнение 18.19. Что, если бы вызов функции `swap()` был бы таким `std::swap(v1.mem1, v2.mem1)?`

18.2.4. Перегрузка и пространства имен

Пространства имен могут повлиять на подбор функции (см. раздел 6.4, стр. 305) двумя способами. Один из них вполне очевиден: объявление или директива `using` может добавить функцию в набор кандидатов. Второй способ менее очевиден.



Зависимый от аргумента поиск и перегрузка

Как упоминалось в предыдущем разделе, поиск имен функций, имеющих один или несколько аргументов типа класса, осуществляется также и в пространстве имен, в котором определен класс каждого аргумента. Это правило влияет также и на выбор кандидатов. Каждое пространство имен, в котором определен класс, используемый в качестве типа параметра (а также те, в которых определены его базовые классы), участвует в поиске функции-кандидата. Все функции этих пространств имен, которые имеют имя, совпадающее с использованным при вызове, будут добавлены в набор кандидатов. Эти функции будут добавлены даже тогда, когда они не видимы в точке обращения:

```
namespace NS {
    class Quote { /* ... */ };
    void display(const Quote&) { /* ... */ }
}
// Базовый класс Bulk_item объявлен в пространстве имен NS
class Bulk_item : public NS::Quote { /* ... */ };
int main() {
    Bulk_item book1;
    display(book1);
    return 0;
}
```

Аргумент `book1` функции `display()` имеет тип класса `Bulk_item`. Функциями-кандидатами для этого вызова функции `display()` будут не только функции с объявлениями, видимыми на момент вызова, но и те, которые объявлены в пространстве имен класса `Bulk_item` и его базового класса `Quote`. Таким образом, функция `display(const Quote&)`, объявленная в пространстве имен `NS`, будет добавлена в набор функций кандидатов.

Перегрузка и объявления using

Чтобы уяснить взаимодействие объявлений `using` и перегрузки, важно помнить, что объявление `using` объявляет только имя, а не конкретную функцию (см. раздел 15.6, стр. 784):

```
using NS::print(int); // ошибка: нельзя указать список параметров
using NS::print;      // ok: в объявлении using указывают только имена
```

Когда объявление `using` используется для функции, все версии этой функции переводятся в текущую область видимости.

Объявление `using` подключает все версии перегруженной функции, чтобы не нарушить интерфейс пространства имен. Ведь предоставляя разные версии функции, автор библиотеки имел на то весомую причину. Разрешив пользователям игнорировать некоторые (но не все) функции из набора перегруженных версий, можно получить довольно странное поведение программы.

Функции, предоставленные объявлением `using`, перегружают любые другие объявления одноименных функций, уже находящихся в данной области видимости.

Если объявление `using` расположено в локальной области видимости, эти имена скрывают существующие объявления для того имени во внешней области видимости. Если объявление `using` вводит функцию в область видимости, в которой уже есть функция с тем же именем и тем же списком параметров, объявление `using` окажется ошибочным. В противном случае объявление `using` создаст дополнительный перегруженный экземпляр данной функции. В результате набор функций-кандидатов увеличится.

Перегрузка и директивы using

Директива `using` переводит члены пространства имен в окружающую область видимости. Если имя функции пространства имен совпадает с именем функции той области видимости, в которую помещено пространство имен, эта функция будет добавлена в набор перегруженных функций.

```
namespace libs_R_us {
    extern void print(int);
    extern void print(double);
}
// обычное объявление
void print(const std::string &);
// директива using добавила имена в набор функций-кандидатов для вызова
// функции print():
using namespace libs_R_us;
// кандидатами на вызов print() в настоящий момент являются:
// print(int) from libs_R_us
// print(double) from libs_R_us
// print(const std::string &) declared explicitly
void fooBar(int ival)
```



```
{
    print("Value: "); // вызов глобальной print(const string &)
    print(ival);      // вызов libs_R_us::print(int)
}
```

В отличие от объявления `using`, не будет ошибки, если директива `using` предоставит функцию с теми же параметрами, что и у существующей функции. Подобно другим конфликтам, вызванным директивами `using`, не будет никаких проблем, если не пытаться вызывать функцию без уточнения, относится ли она к пространству имен или к текущей области видимости.

Перегрузка при нескольких директивах `using`

Если в коде присутствует несколько директив `using`, частью набора функций-кандидатов станут соответствующие функции из каждого пространства имен.

```
namespace AW {
    int print(int);
}
namespace Primer {
    double print(double);
}
// директивы using создают набор перегруженных функций из разных
// пространств имен
using namespace AW;
using namespace Primer;
long double print(long double);
int main() {
    print(1); // вызов AW::print(int)
    print(3.1); // вызов Primer::print(double)
    return 0;
}
```

Набор перегруженных функций `print()` в глобальной области видимости содержит функции `print(int)`, `print(double)` и `print(long double)`. Все они составят набор перегруженных функций, рассматриваемых при вызове функции `print()` в функции `main()`, даже в том случае, если первоначально эти функции были объявлены в различных областях видимости пространства имен.

Упражнения раздела 18.2.4

Упражнение 18.20. С учетом следующего кода укажите, какие из функций (если они есть) соответствуют обращению к функции `compute()`. Перечислите функции-кандидаты и подходящие функции. Какая последовательность преобразований типов (если есть) будет применена к аргументу, чтобы он точно соответствовал параметру каждой подходящей функции?

```
namespace primerLib {
    void compute();
    void compute(const void *);
}
```

```
using primerLib::compute;
void compute(int);
void compute(double, double = 3.4);
void compute(char*, char* = 0);
void f()
{
    compute(0);
}
```

Что произойдет в случае, если объявления `using` будут расположены в функции `main()` перед обращением к функции `compute()`? Ответьте на те же вопросы, что и в предыдущем упражнении.

18.3. Множественное и виртуальное наследование

Множественное наследование (multiple inheritance) — это способность получить класс как производный непосредственно от нескольких базовых классов (см. раздел 15.2.2, стр. 752). Полученный в результате класс наследует свойства всех своих базовых классов. Несмотря на простоту концепции, одновременное использование нескольких базовых классов может создать достаточно много сложностей как на этапе проектирования, так и на этапе реализации.

Для исследования множественного наследования используем пример иерархии из животного мира. Животные расположены на разных уровнях абстракции. Есть индивидуальные животные, различающиеся по именам, такие как Ling-ling (Линг-линг), Mowgli (Маутли) и Balou (Балу). Каждое животное можно отнести к определенному виду; Линг-линг, например, это гигантская панда. Виды в свою очередь относятся к определенным семействам. Гигантская панда принадлежит к семейству медведей, а каждое семейство является членом сообщества животного мира.

Каждый уровень абстракции содержит разнообразные данные и функции. Определим класс `ZooAnimal` как абстрактный, призванный содержать информацию, которая является общей для всех животных и предоставляет открытый интерфейс. Класс `Bear` (Медведь) будет содержать информацию, которая является специфической для семейства медведей, и т.д.

Кроме классов животных, здесь можно определить дополнительные классы, которые инкапсулируют различные абстракции, например, животных, подвергающихся опасности. В данной реализации класс `Panda` (Панда) будет получен в результате множественного наследования от классов `Bear` и `Endangered` (Подвергающийся опасности).

18.3.1. Множественное наследование

Список наследования производного класса может содержать несколько базовых классов:

```
class Bear : public ZooAnimal {  
class Panda : public Bear, public Endangered { /* ... */ };
```

У каждого базового класса есть необязательный спецификатор доступа (см. раздел 15.5, стр. 773). Как обычно, если спецификатор доступа отсутствует, по умолчанию подразумевается спецификатор `private` (закрытый), если используется ключевое слово `class`, и `public` (открытый), если используется ключевое слово `struct` (см. раздел 15.5, стр. 777).

Как и при одиночным наследовании, список наследования может включить только те классы, которые были определены и не были определены как `final` (см. раздел 15.2.2, стр. 752). Язык C++ не налагает никаких ограничений на количество базовых классов, из которых может быть получен производный класс. Однако базовый класс может присутствовать в списке наследования только один раз.

При множественном наследовании классы наследуют состояние каждого из базовых классов

При множественном наследовании объект производного класса внутренне содержит объекты каждого из своих базовых классов (см. раздел 15.2.2, стр. 754). Например, на рис. 18.2 у объекта `Panda` есть часть класса `Bear` (которая сама содержит часть `ZooAnimal`), часть класса `Endangered` и нестатические переменные-члены, если таковые имеются, объявленные в пределах класса `Panda`.

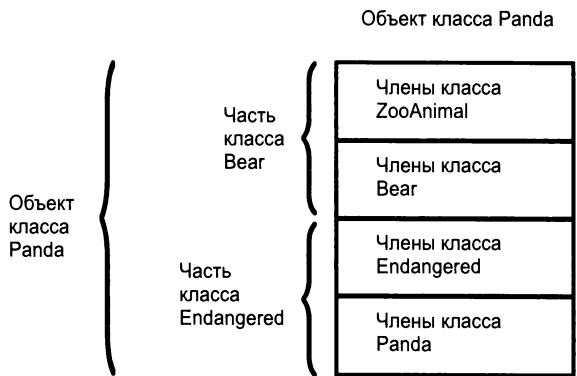


Рис. 18.2. Концептуальная структура объекта класса `Panda`

Конструкторы производного класса инициализируют все объекты базовых классов

Создание объекта производного класса подразумевает создание и инициализацию внутренних объектов всех его базовых классов. В случае одиночного наследования из (единого) базового класса (см. раздел 15.2.2, стр. 755) в списке инициализации конструктора производного класса можно передать значения только для прямых базовых классов:

```
// явная инициализация объектов обоих базовых классов
Panda::Panda(std::string name, bool onExhibit)
    : Bear(name, onExhibit, "Panda"),
      Endangered(Endangered::critical) { }
// неявное применение стандартного конструктора класса Bear для
// инициализации его внутреннего объекта
Panda::Panda()
    : Endangered(Endangered::critical) { }
```

Список инициализации конструктора позволяет передать аргументы каждому из прямых базовых классов, однако *порядок выполнения конструкторов* (constructor order) зависит от порядка их расположения в списке наследования класса. Порядок их расположения в списке инициализации конструктора не имеет значения. Объект класса Panda инициализируется следующим образом.

- Внутренний объект класса ZooAnimal, самого первого базового класса иерархии класса Panda, непосредственного базового для класса Bear создается первым.
- Внутренний объект класса Bear, первого непосредственного базового класса для класса Panda, инициализируется следующим.
- Внутренний объект класса Endangered, второго непосредственного базового класса для класса Panda, инициализируется следующим.
- Последней инициализируется наиболее производная часть класса Panda.

Унаследованные конструкторы и множественное наследование

C++ 11 По новому стандарту производный класс может наследовать свои конструкторы от одного или нескольких своих базовых классов (см. раздел 15.7.4, стр. 793). Нельзя наследовать тот же конструктор (т.е. конструктор с тем же списком параметров) от более чем одного базового класса:

```
struct Base1 {
    Base1() = default;
    Base1(const std::string&);
    Base1(std::shared_ptr<int>);
};
struct Base2 {
    Base2() = default;
```

```

    Base2(const std::string&);
    Base2(int);
};
// ошибка: D1 пытается унаследовать D1::D1(const string&) от обоих
// базовых классов
struct D1: public Base1, public Base2 {
    using Base1::Base1; // наследует конструкторы от Base1
    using Base2::Base2; // наследует конструкторы от Base2
};

```

Класс, унаследовавший тот же конструктор от нескольких базовых классов, должен определить собственную версию этого конструктора:

```

struct D2: public Base1, public Base2 {
    using Base1::Base1; // наследует конструкторы от Base1
    using Base2::Base2; // наследует конструкторы от Base2
    // D2 должен определить собственный конструктор, получающий string
    D2(const string &s): Base1(s), Base2(s) { }
    D2() = default; // необходимо, поскольку D2 определяет собственный
                  // конструктор
};

```

Деструкторы и множественное наследование

Как обычно, деструктор в производном классе отвечает за освобождение ресурсов, зарезервированных этим классом. Автоматически освобождаются члены только производного класса и всех базовых классов. Тело синтезируемого деструктора пусто.

Деструкторы всегда выполняются в порядке, обратном вызову конструкторов. В данном примере порядок вызова деструкторов будет следующим: `~Panda()`, `~Endangered()`, `~Bear()`, `~ZooAnimal()`.

Функции копирования и перемещения при множественном наследовании

Как и в случае одиночного наследования, классы с несколькими базовыми классами, определяющими собственные конструкторы копирования, перемещения и операторы присвоения, должны копировать, перемещать и присваивать весь объект (см. раздел 15.7.2, стр. 786). Базовые части класса, производного от нескольких базовых, автоматически копируются, перемещаются и присваиваются, только если производный класс использует синтезируемые версии этих функций-членов. В синтезируемых функциях-членах управления копированием каждый базовый класс неявно создается, присваивается или удаляется с использованием соответствующего члена базового класса.

Например, если класс `Panda` использует синтезируемые функции-члены, то инициализация объекта `ling_ling` вызовет конструктор копий класса `Bear`, который в свою очередь вызовет конструктор копий класса `ZooAnimal` прежде, чем выполнить конструктор копий класса `Bear`:

```
Panda ying_yang("ying_yang");
Panda ling_ling = ying_yang; // использует конструктор копий
```

Как только часть Bear объекта ling_ling создана, выполняется конструктор копий класса Endangered, создающий соответствующую часть объекта. И наконец, выполняется конструктор копий класса Panda. Аналогично для синтезируемого конструктора перемещения.

Синтезируемый оператор присвоения копии ведет себя так же, как и конструктор копий. Сначала он присваивает часть Bear (и его часть ZooAnimal) объекта, затем часть Endangered и наконец часть Panda. Оператор присвоения при перемещении ведет себя подобным образом.

Упражнения раздела 18.3.1

Упражнение 18.21. Объясните следующие объявления. Найдите все ошибки и объясните их причину:

```
(a) class CADVehicle : public CAD, Vehicle { ... };
(b) class DbList: public List, public List { ... };
(c) class iostream: public istream, public ostream { ... };
```

Упражнение 18.22. С учетом следующей иерархии класса, в которой у каждого класса определен стандартный конструктор:

```
class A { ... };
class B : public A { ... };
class C : public B { ... };
class X { ... };
class Y { ... };
class Z : public X, public Y { ... };
class MI : public C, public Z { ... };
```

Каков порядок выполнения конструкторов при создании следующего объекта?

```
MI mi;
```

18.3.2. Преобразования и несколько базовых классов

При одиночном наследовании указатель или ссылка на производный класс могут быть автоматически преобразованы в указатель или ссылку на базовый класс (см. раздел 15.2.2, стр. 754 и раздел 15.5, стр. 774). Это справедливо и для множественного наследования. Указатель или ссылка на производный класс могут быть преобразованы в указатель или ссылку на любой из его базовых классов. Например, указатель или ссылка на класс ZooAnimal, Bear или Endangered может указывать или ссылаться на объект класса Panda.

```
// функции, получающие ссылки на класс, базовый для класса Panda
void print(const Bear&);
void highlight(const Endangered&);
ostream& operator<<(ostream&, const ZooAnimal&);
```

```
Panda ying_yang("ying_yang");
print(ying_yang);           // передает объект класса Panda как
                             // ссылку на объект класса Bear
highlight(ying_yang);       // передает объект класса Panda как
                             // ссылку на объект класса Endangered
cout << ying_yang << endl;  // передает объект класса Panda как
                             // ссылку на объект класса ZooAnimal
```

Компилятор даже не пытается как-то различать базовые классы. Преобразования в каждый из базовых классов происходят одинаково успешно. Рассмотрим, например, перегруженную версию функции `print()`:

```
void print(const Bear&);
void print(const Endangered&);
```

Вызов функции `print()` без квалификации для объекта класса `Panda` приводит к ошибке во время выполнения.

```
Panda ying_yang("ying_yang");
print(ying_yang); // ошибка: неоднозначность
```

Поиск на основании типа указателя или ссылки

Как и при одиночном наследовании, статический тип объекта, указателя или ссылки определяет, какие из членов можно использовать. Если используется указатель класса `ZooAnimal`, для применения будут пригодны только те функции, которые определены в этом классе. Части интерфейса класса `Panda`, специфические для классов `Bear`, `Panda` и `Endangered`, окажутся недоступны. Аналогично указатель или ссылка на класс `Bear` применимы только для доступа к членам классов `Bear` и `ZooAnimal`, а указатель или ссылка на класс `Endangered` ограничены лишь членами класса `Endangered`.

В качестве примера рассмотрим следующие вызовы с учетом того, что эти классы определяют виртуальные функции, перечисленные в табл. 18.1.

```
Bear *pb = new Panda("ying_yang");
pb->print();      // ok: Panda::print()
pb->cuddle();     // ошибка: не является частью интерфейса Bear
pb->highlight();  // ошибка: не является частью интерфейса Bear
delete pb;       // ok: Panda::~~Panda()
```

Когда объект класса `Panda` используется при помощи указателя или ссылки на класс `Endangered`, части объекта класса `Panda`, специфические для классов `Panda` и `Bear`, становятся недоступными.

```
Endangered *pe = new Panda("ying_yang");
pe->print();     // ok: Panda::print()
pe->toes();      // ошибка: не является частью интерфейса Endangered
pe->cuddle();    // ошибка: не является частью интерфейса Endangered
pe->highlight(); // ok: Panda::highlight()
delete pe;      // ok: Panda::~~Panda()
```

Таблица 18.1. Виртуальные функции иерархии классов ZooAnimal/Endangered

Функция	Класс, определяющий собственную версию
print()	ZooAnimal::ZooAnimal Bear::Bear Endangered::Endangered Panda::Panda
highlight	Endangered::Endangered Panda::Panda
toes	Bear::Bear Panda::Panda
cuddle	Panda::Panda
Деструктор	ZooAnimal::ZooAnimal Endangered::Endangered

Упражнения раздела 18.3.2

Упражнение 18.23. Используя иерархию из упражнения 18.22, а также определенный ниже класс D и с учетом наличия у каждого класса стандартного конструктора, укажите, какие из следующих преобразований недопустимы (если таковые вообще имеются)?

```
class D : public X, public C { ... };
D *pd = new D;
(a) X *px = pd;      (b) A *pa = pd;
(c) B *pb = pd;      (d) C *pc = pd;
```

Упражнение 18.24. На стр. 1007 представлена последовательность вызовов через указатель на класс Bear, указывающих на объект класса Panda. Объясните каждый вызов, подразумевая, что вместо него используется указатель на класс ZooAnimal, указывающий на объект класса Panda.

Упражнение 18.25. Предположим, существуют два базовых класса, Base1 и Base2, в каждом из которых определена виртуальная функция-член по имени print() и виртуальный деструктор. От этих базовых классов были получены следующие классы, в каждом из которых переопределена функция print().

```
class D1 : public Base1 { /* ... */ };
class D2 : public Base2 { /* ... */ };
class MI : public D1, public D2 { /* ... */ };
```


Используя следующие определения, укажите, какая из функций используется при каждом вызове:

```
Base1 *pb1 = new MI;  
Base2 *pb2 = new MI;  
D1 *pd1 = new MI;  
D2 *pd2 = new MI;  
(a) pb1->print();    (b) pd1->print();    (c) pd2->print();  
(d) delete pb2;      (e) delete pd1;    (f) delete pd2;
```

18.3.3. Область видимости класса при множественном наследовании

При одиночном наследовании область видимости производного класса вкладывается в пределы его прямых и косвенных базовых классов (см. раздел 15.6, стр. 778). Поиск имен осуществляется по всей иерархии наследования. Имена, определенные в производном классе, скрывают совпадающие имена в базовом классе.

При множественном наследовании поиск осуществляется *одновременно* во всех прямых базовых классах. Если имя находится в нескольких базовых классах, происходит ошибка неоднозначности.

В рассматриваемом примере, если имя используется через указатель, ссылку или объект класса Panda, деревья иерархии Endangered и Bear/ZooAnimal исследуются параллельно. Если имя находится в нескольких иерархиях, то возникнет неоднозначность. Для класса вполне допустимо наследовать несколько членов с тем же именем. Но если это имя необходимо использовать, следует указать, какая именно версия имеется в виду.



Когда у класса есть несколько базовых классов, производный класс вполне может унаследовать одноименный член от двух и более своих базовых классов. При использовании этого имени без уточнения класса происходит неоднозначность.

Например, если классы ZooAnimal и Endangered определяют функцию-член `max_weight()`, а класс Panda не определяет ее, то следующий вызов ошибочен:

```
double d = ying_yang.max_weight();
```

В результате наследования класс Panda получает две функции-члена `max_weight()`, что совершенно допустимо. Наследование создает *потенциальную* неоднозначность. Ее вполне можно избежать, если объект Panda не будет вызывать функцию-член `max_weight()`. Ошибки также можно избежать, если явно указать требуемую версию функции: `ZooAnimal::max_weight()` или `Endangered::max_weight()`. Ошибка неоднозначности произойдет только при попытке использования функции без уточнения.

Неоднозначность двойного наследования функции-члена `max_weight` вполне очевидна и логична. Удивительно узнать то, что ошибка произошла бы, даже если у двух наследованных функций были разные списки параметров. Точно так же эта ошибка произошла бы даже в случае, если бы функция `max_weight()` была закрытой в одном классе и открытой или защищенной в другом. И наконец, если бы функция `max_weight()` была определена в классе `Bear`, а не в классе `ZooAnimal`, то вызов все равно был бы ошибочен.

Как обычно, поиск имени осуществляется под контролем соответствия типов (см. раздел 6.4.1, стр. 307). Когда компилятор находит имя функции `max_weight()` в двух разных областях видимости, он оповещает об ошибке неоднозначности.

Проще всего избежать потенциальных неоднозначностей, определив версию такой функции в производном классе. Например, снабдив класс `Panda` функцией `max_weight()`, можно решить все проблемы:

```
double Panda::max_weight() const
{
    return std::max(ZooAnimal::max_weight(),
                   Endangered::max_weight());
}
```

Упражнения раздела 18.3.3

Упражнение 18.26. С учетом иерархии на стр. 1011 объясните, почему ошибочен следующий вызов функции `print()`? Исправьте структуру `MI` так, чтобы позволить этот вызов.

```
MI mi;
mi.print(42);
```

Упражнение 18.27. С учетом иерархии на стр. 1011 и того, что в структуру `MI` добавлена приведенная ниже функция `foo()`, ответьте на следующие вопросы:

```
int ival;
double dval;
void MI::foo(double cval)
{
    int dval;
    // варианты вопросов упражнения располагаются здесь ...
}
```

- Перечислите все имена, видимые из функции `MI::foo()`.
- Видимы ли какие-нибудь имена из больше чем одного базового класса?
- Присвойте локальному экземпляру переменной `dval` сумму переменных-членов `dval` объектов классов `Base1` и `Derived`.
- Присвойте значение последнего элемента вектора `MI::dvec` переменной-члену `Base2::fval`.
- Присвойте переменной-члену `cval` класса `Base1` первый символ строки `sval` класса `Derived`.

Код для упражнений раздела 18.3.3

```
struct Base1 {
    void print(int) const; // по умолчанию открыты
protected:
    int ival;
    double dval;
    char cval;
private:
    int *id;
};

struct Base2 {
    void print(double) const; // по умолчанию открыты
protected:
    double fval;
private:
    double dval;
};

struct Derived : public Base1 {
    void print(std::string) const; // по умолчанию открыты
protected:
    std::string sval;
    double dval;
};

struct MI : public Derived, public Base2 {
    void print(std::vector<double>); // по умолчанию открыты
protected:
    int *ival;
    std::vector<double> dvec;
};
```

18.3.4. Виртуальное наследование

Хотя список наследования класса не может включать тот же базовый класс несколько раз, класс вполне может унаследовать тот же базовый класс многократно. Тот же базовый класс может быть унаследован косвенно, от двух его собственных прямых базовых классов, либо он может унаследовать некий класс и прямо, и косвенно, через другой из его базовых классов.

Например, библиотечные классы ввода-вывода `istream` и `ostream` происходят от общего абстрактного базового класса `basic_ios`. Этот класс содержит буфер потока и управляет флагом состояния потока. Класс `iostream`, способный и читать, и писать в поток, происходит непосредственно и от класса `istream`, и от класса `ostream`. Поскольку оба класса происходят от класса `basic_ios`, класс `iostream` наследует этот базовый класс дважды: один раз от класса `istream` и один раз от класса `ostream`.

По умолчанию объект производного класса содержит отдельные части, соответствующие каждому классу в его цепи наследования. Если тот же базовый класс наследуется несколько раз, то у объекта производного класса будет больше одного внутреннего объекта этого типа.

Для такого класса, как `iostream`, это стандартное поведение не работает. Объект класса `iostream` должен использовать тот же буфер и для чтения, и для записи, а его флаг должен отражать состояние операций и ввода, и вывода. Если у объекта класса `iostream` будут две копии объекта класса `basic_ios`, то их совместное использование невозможно.

В языке C++ для решения этой проблемы используется *виртуальное наследование* (virtual inheritance). Виртуальное наследование позволяет классу указать, что его базовый класс будет использоваться совместно. Совместно используемый внутренний объект базового класса называется *виртуальным базовым классом* (virtual base class). Независимо от того, сколько раз тот же базовый виртуальный класс присутствует в иерархии наследования, объект производного класса содержит только один совместно используемый внутренний объект этого виртуального базового класса.

Разные классы Panda

В прошлом велись дебаты о принадлежности вида *панда* к семейству енотов или медведей. Чтобы отобразить эти сомнения, изменим класс `Panda` так, чтобы он происходил и от класса `Bear`, и от класса `Raccoon`. Чтобы избавить класс `Panda` от двух частей базового класса `ZooAnimal`, определим наследование классов `Bear` и `Raccoon` от класса `ZooAnimal` как виртуальное. Новая иерархия представлена на рис. 18.3.

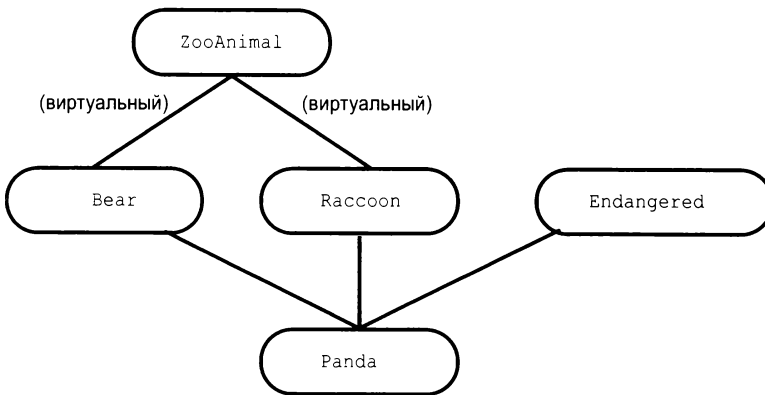


Рис. 18.3. Виртуальное наследование в иерархии класса `Panda`

Глядя на новую иерархию, можно заметить неочевидный аспект виртуального наследования. Виртуальное наследование должно быть осуществлено прежде, чем в нем возникнет потребность. Например, в этих классах потребность в виртуальном наследовании возникает только при определении класса `Panda`. Но если бы классы `Bear` и `Raccoon` не определили бы свое происхождение от класса `ZooAnimal` как виртуальное, конструкция класса `Panda` была бы неудачна.

На практике необходимость наличия промежуточного базового класса при виртуальном наследовании редко создает проблемы. Обычно иерархия классов, в которой используется виртуальное наследование, разрабатывается сразу и одним лицом (или группой разработчиков). Ситуации, когда разработку виртуального базового класса необходимо поручить независимому производителю, чрезвычайно редки, а разработчик нового базового класса не может внести изменения в существующую иерархию.



Виртуальное наследование влияет на те классы, которые происходят от виртуального базового класса впоследствии; оно не влияет на класс производный непосредственно.

Использование виртуального базового класса

Базовый класс объявляется виртуальным при помощи ключевого слова `virtual` в списке наследования:

```
// порядок расположения ключевых слов public и virtual несуществен
class Raccoon : public virtual ZooAnimal { /* ... */ };
class Bear : virtual public ZooAnimal { /* ... */ };
```

Здесь класс `ZooAnimal` объявлен виртуальным базовым для классов `Bear` и `Raccoon`.

Спецификатор `virtual` заявляет о готовности совместно использовать единый экземпляр указанного базового класса в последующих производных классах. Нет никаких особых ограничителей на классы, используемые как виртуальные базовые классы.

Для наследования от класса, имеющего виртуальный базовый класс, не нужно ничего особенного:

```
class Panda : public Bear,
              public Raccoon, public Endangered {
};
```

Здесь класс `Panda` наследует класс `ZooAnimal` через два своих базовых класса — `Raccoon` и `Bear`. Но поскольку эти классы происходят от класса `ZooAnimal` виртуально, у класса `Panda` есть только одна часть базового класса `ZooAnimal`.

Для базовых классов поддерживаются стандартные преобразования

Объектом производного класса можно манипулировать как обычно, при помощи указателя или ссылки на базовый класс, хотя он и является виртуальным. Например, все следующие преобразования для базового класса объекта класса `Panda` вполне допустимы:

```
void dance(const Bear&);
void rummage(const Raccoon&);
```

```
ostream& operator<<(ostream&, const ZooAnimal&);
Panda ying_yang;
dance(ying_yang);    // ok: передает объект Panda как Bear
rummage(ying_yang);  // ok: передает объект Panda как Raccoon
cout << ying_yang;   // ok: передает объект Panda как ZooAnimal
```

Видимость членов виртуальных базовых классов

Поскольку виртуальному базовому классу соответствует только один совместно используемый внутренний объект, к членам объекта этого базового класса можно обратиться непосредственно и однозначно. Кроме того, если член виртуального базового класса переопределяется только в одной ветви наследования, к этому переопределенному члену класса можно обратиться непосредственно. Если член переопределяется больше чем одним базовым классом, то производный класс вообще должен определить собственную версию этого члена.

Предположим, например, что класс В определяет члены по имени *x*; класс D1 виртуально наследует класс В, как и класс D2; а класс D происходит от классов D1 и D2. Из области видимости класса D член *x* видим через оба своих базовых класса. Есть три возможности использовать член *x* через объект класса D:

- Если член *x* не будет определен ни в классе D1, ни в D2, то будет использован член класса В; никакой неоднозначности нет. Объект класса D содержит только один экземпляр члена *x*.
- Если *x* является членом класса В и одного (но не обоих) из классов D1 или D2, никакой неоднозначности снова нет: версия в производном классе имеет приоритет перед совместно используемым виртуальным базовым классом В.
- Если член *x* определяется и в классе D1, и в классе D2, то прямой доступ к этому члену неоднозначен.

Как и в иерархии с неvirtуальным множественным наследованием, подобная неоднозначность лучше всего устраняется переопределением члена в производном классе.

Упражнения раздела 18.3.4

Упражнение 18.28. Рассмотрим следующую иерархию класса. Можно ли в классе VMI обращаться к унаследованным членам без уточнения? Какие из них требуют полностью квалифицированных имен? Объясните, почему.

```
struct Base {
    void bar(int); // по умолчанию открыты
protected:
    int ival;
};
```

```

struct Derived1 : virtual public Base {
    void bar(char); // по умолчанию открыты
    void foo(char);
protected:
    char cval;
};
struct Derived2 : virtual public Base {
    void foo(int); // по умолчанию открыты
protected:
    int ival;
    char cval;
};
class VMI : public Derived1, public Derived2 { };

```

18.3.5. Конструкторы и виртуальное наследование

При виртуальном наследовании виртуальный базовый класс инициализируется конструктором самого последнего производного класса. В рассматриваемом примере при создании объекта класса `Panda` инициализацию членов базового класса `ZooAnimal` контролирует конструктор класса `Panda`.

Чтобы понять это правило, рассмотрим происходящее при применении обычных правил инициализации. В этом случае объект виртуального базового класса мог бы быть инициализирован несколько раз. Он был бы инициализирован вдоль каждой ветви наследования, содержащей этот виртуальный базовый класс. В данном примере, если бы к классу `ZooAnimal` применялись обычные правила инициализации, то части `Bear` и `Raccoon` инициализировали бы часть `ZooAnimal` объекта класса `Panda`.

Конечно, каждый базовый класс в иерархии объекта мог бы в некоторый момент быть “более производным”. Поскольку вполне можно создавать независимые объекты класса, производного от виртуального базового класса, конструкторы в этом классе должны инициализировать его виртуальный базовый класс. Например, когда в рассматриваемой иерархии создается объект класса `Bear` (или `Raccoon`), никакого дальнейшего применения производного класса нет. В данном случае конструкторы класса `Bear` (или `Raccoon`) непосредственно инициализируют базовую часть `ZooAnimal`, как обычно:

```

Bear::Bear(std::string name, bool onExhibit):
    ZooAnimal(name, onExhibit, "Bear") { }
Raccoon::Raccoon(std::string name, bool onExhibit)
    : ZooAnimal(name, onExhibit, "Raccoon") { }

```

Когда создается объект класса `Panda`, он является наиболее производным типом и контролирует инициализацию совместно используемого базового класса `ZooAnimal`. Даже при том, что класс `ZooAnimal` не является прямым базовым классом для класса `Panda`, часть `ZooAnimal` инициализирует конструктор класса `Panda`:

```
Panda::Panda(std::string name, bool onExhibit)
    : ZooAnimal(name, onExhibit, "Panda"),
      Bear(name, onExhibit),
      Raccoon(name, onExhibit),
      Endangered(Endangered::critical),
      sleeping_flag(false) { }
```

Как создается объект при виртуальном наследовании

Порядок создания объекта с виртуальным базовым классом немного отличается от обычного: сначала инициализируется часть виртуального базового класса с использованием инициализаторов, предоставленных в конструкторе для наиболее производного класса. Как только создана часть виртуального базового класса, создаются части прямых базовых классов в порядке их расположения в списке наследования.

Например, объект класса `Panda` создается так.

- Сначала создается часть виртуального базового класса `ZooAnimal`. При этом используются инициализаторы из списка инициализации конструктора класса `Panda`.
- Затем создается часть `Bear`.
- Затем создается часть `Raccoon`.
- Следующей создается часть прямого базового класса `Endangered`.
- Наконец создается часть `Panda`.

Если конструктор класса `Panda` не инициализирует явно часть базового класса `ZooAnimal`, будет использован стандартный конструктор класса `ZooAnimal`. Если у класса `ZooAnimal` нет стандартного конструктора, произойдет ошибка.



Части виртуальных базовых классов всегда создаются до частей обычных базовых классов, независимо от того, где они располагаются в иерархии наследования.

Порядок выполнения конструкторов и деструкторов

У класса может быть несколько виртуальных базовых классов. В этом случае части виртуальных классов создаются в порядке их расположения в списке наследования. Например, в следующей иерархии наследования у класса `TeddyBear` (МедвежонокТедди) есть два виртуальных базовых класса: прямой виртуальный базовый класс `ToyAnimal` (ИгрушечноеЖивотное) и косвенный базовый класс `ZooAnimal`, от которого происходит класс `Bear`:

```
class Character { /* ... */ };
class BookCharacter : public Character { /* ... */ };
class ToyAnimal { /* ... */ };
class TeddyBear : public BookCharacter,
public Bear, public virtual ToyAnimal
    { /* ... */ };
```


Чтобы выявить наличие виртуальных базовых классов, прямые базовые классы просматриваются в порядке объявления. Если это так, то сначала создаются части виртуальных базовых классов, затем выполняются конструкторы обычных, не виртуальных базовых классов в порядке их объявления. Таким образом, чтобы создать объект класса `TeddyBear`, конструкторы его частей вызываются в следующем порядке:

```
ZooAnimal();      // виртуальный базовый класс Bear
ToyAnimal();      // прямой виртуальный базовый класс
Character();      // косвенный базовый класс первого не виртуального
                  // базового класса
BookCharacter();  // первый прямой не виртуальный базовый класс
Bear();           // второй прямой не виртуальный базовый класс
TeddyBear();      // наиболее производный класс
```

Тот же порядок создания используется в синтезируемом конструкторе копий и конструкторах перемещения, в синтезируемых операторах присвоения члены присваиваются в том же порядке. Вызов деструкторов базовых классов осуществляется в порядке, обратном порядку вызова конструкторов. Часть `TeddyBear` будет удалена сначала, а часть `ZooAnimal` — последней.

Упражнения раздела 18.3.5

Упражнение 18.29. Имеется следующая иерархия классов:

```
class Class { ... };
class Base : public Class { ... };
class D1 : virtual public Base { ... };
class D2 : virtual public Base { ... };
class MI : public D1, public D2 { ... };
class Final : public MI, public Class { ... };
```

- Каков порядок вызова конструкторов и деструкторов объектов класса `Final`?
- Сколько внутренних объектов класса `Base` находится в объекте класса `Final`? А сколько внутренних объектов класса `Class`?
- Какие из следующих случаев присвоения приведут к ошибке во время компиляции?

```
Base *pb; Class *pc; MI *pmi; D2 *pd2;
(a) pb = new Class;      (b) pc = new Final;
(c) pmi = pb;            (d) pd2 = pmi;
```

Упражнение 18.30. Определите в классе `Base` стандартный конструктор, конструктор копий и конструктор с параметром типа `int`. Определите те же три конструктора в каждом производном классе. Каждый конструктор должен использовать свой аргумент для инициализации своей части `Base`.

Резюме

Язык C++ применяется для решения широкого диапазона проблем: от требующих лишь нескольких часов работы до занимающих годы работы больших групп разработчиков. Некоторые из средств языка C++ наиболее полезны при создании крупномасштабных приложений. Имеется в виду обработка исключений, пространства имен и множественное или виртуальное наследование.

Обработка исключений позволяет отделить ту часть кода, где может произойти ошибка, от той части кода, где она обрабатывается. При передаче исключения выполнение текущей функции приостанавливается и начинается поиск ближайшей директивы `catch`. Локальные переменные, определенные в покидаемых при поиске директив `catch` функциях, удаляются в ходе обработки исключения.

Пространства имен — это механизм управления большими и сложными приложениями, формируемыми из кода, созданного независимыми поставщиками. Пространство имен является областью видимости, в которой могут быть определены объекты, типы, функции, шаблоны и другие пространства имен. Стандартная библиотека определена в пространстве имен `std`.

С концептуальной точки зрения множественное наследование — довольно простое понятие: производный класс может быть унаследован от нескольких прямых базовых классов. Объект производного класса состоит из частей, представляющих собой внутренние объекты всех своих базовых классов. Концепция действительно проста, но на практике сопряжена со многими сложностями. В частности, наследование от нескольких базовых классов создает вероятность конфликтов имен и в результате порождает неоднозначные обращения к именам из базовых частей объекта.

Если класс происходит от нескольких непосредственных базовых классов, не исключена ситуация, когда эти классы сами могут иметь общий базовый класс. В таких случаях промежуточные классы могут применить виртуальное наследование, позволяющее другим классам иерархии, унаследовавшим тот же базовый класс, совместно использовать его внутренний объект. Таким образом, объект производного класса будет иметь только одну совместно используемую копию внутреннего объекта виртуального базового класса.

Термины

Безымянное пространство имен (`unnamed namespace`). Пространство имен, определенное без имени. К именам, определенным в безымянном пространстве имен, можно обращаться непосредственно, без оператора области видимости. Каждый файл имеет собственное, уникальное безымянное пространство имен. Имена в файле невидимы вне данного файла.

- Блок try (try block).** Блок операторов, начинающийся ключевым словом `try` и содержащий одну или несколько директив `catch`. Если код в блоке `try` передает исключение и одна из директив `catch` соответствует типу переданного исключения, то переданное исключение будет обработано этим обработчиком. В противном случае исключение будет передано из блока `try` другому обработчику, далее по цепи вызовов.
- Блок try функции (function try block).** Используется для обработки исключений из списка инициализации конструктора. Ключевое слово `try` располагается перед двоеточием, начинающим список инициализации конструктора (или перед открывающей фигурной скобкой тела конструктора, если список инициализации пуст), и завершается одной или несколькими директивами `catch`, которые следуют после закрывающей фигурной скобки тела конструктора.
- Виртуальное наследование (virtual inheritance).** Форма множественного наследования, при котором производные классы совместно используют одну копию экземпляра базового класса, даже если в иерархии он встречается несколько раз.
- Виртуальный базовый класс (virtual base class).** Базовый класс, при наследовании которого было использовано ключевое слово `virtual`. В объекте производного класса часть виртуального базового класса содержится только в одном экземпляре, даже если в иерархии этот класс присутствует несколько раз. При не виртуальном наследовании конструктор может инициализировать только непосредственный базовый класс (классы). При виртуальном наследовании этот класс мог бы быть инициализирован несколькими производными классами, которые должны предоставить инициализирующие значения для всех его виртуальных предков.
- Выражение throw e (передача исключения).** Выражение, которое прерывает текущий поток выполнения. Каждый оператор `throw` передает управление ближайшему окружающему блоку `catch`, который способен обработать исключение переданного типа. Выражение `e` будет скопировано в объект исключения.
- Глобальное пространство имен (global namespace).** Неявное пространство имен, содержащее все определения глобальных объектов, которыми обладает каждая программа.
- Директива catch (catch clause).** Часть программы, которая обрабатывает исключение. Директива обработчика состоит из ключевого слова `catch`, за которым следуют объявление исключения и блок операторов. Код в блоке `catch` предназначен для обработки исключений типа, указанного в его объявлении.
- Директива using (using directive).** Объявление в форме `using NS;` делает *все* имена пространства имен `NS` доступными в ближайшей области видимости, содержащей и директиву `using`, и само пространство имен.
- Загромождение пространства имен (namespace pollution).** Термин, используемый для описания ситуации, когда все имена классов и функций располагаются в глобальном пространстве имен. Большие программы, использующие код, который создан несколькими независимыми производителями, зачастую сталкиваются с конфликтами имен, если эти имена глобальны.
- Множественное наследование (multiple inheritance).** Наследование, при котором класс имеет несколько непосредственных базовых классов. Производный класс наследует члены всех своих базовых классов. Имена нескольких базовых классов указываются в списке наследования класса. Для каждого базового класса может быть предоставлен отдельный спецификатор доступа.

Обработка исключений (exception handling). Механизм уровня языка, предназначенный для ликвидации аномалий времени выполнения. Один независимо разработанный раздел кода может обнаружить проблему и передать исключение, которое может получить и обработать другая независимо разработанная часть программы. Часть кода, обнаруживающая ошибку, передает исключение, а часть кода, получающая его, осуществляет обработку.

Обработчик (handler). Синоним директивы `catch`.

Обработчик для всех исключений (catch-all). Директива `catch`, в которой объявляется исключение. Директива обработчика для всех исключений обрабатывает исключения любого типа. Обычно он используется для предварительной обработки исключения, осуществляемой локально. Затем исключение повторно передается другой части программы, в которой и осуществляется устранение причины проблем.

Объект исключения (exception object). Объект, используемый для передачи сообщения между блоками `throw` и `catch`. Объект создается в точке передачи и является копией использованного выражения. Объект исключения существует, пока не сработает последний обработчик для его типа. Тип объекта соответствует типу использованного выражения.

Объявление `using` (using declaration). Механизм, позволяющий ввести одно имя из пространства имен в текущую область видимости. `using std::cout;` Это объявление сделает имя `cout` из пространства имен `std` доступным в текущей области видимости, благодаря чему имя `cout` можно применять без спецификатора `std::`.

Объявление исключения (exception declaration). Объявление директивы `catch`, определяющее тип обрабатываемого исключения. Объявление действует как список параметров, каждый параметр которого инициализируется объектом исключения. Если спецификатор исключения имеет не ссылочный тип, то объект исключения копируется в обработчик.

Оператор `noexcept`. Оператор, возвращающий тип `bool` и указывающий, способно ли данное выражение передать исключение. Выражение не вычисляется. Результат — константное выражение. Его значение `true`, если выражение не содержит оператора `throw` и вызывает только те функции, которые не передают исключений; в противном случае результат — `false`.

Оператор области видимости (scope operator). Оператор `::` используется для доступа к именам пространства имен или класса.

Передача (raise). Синоним термина “throw” (передача). Программисты C++ используют термины “throwing” и “raising” как синонимы, означающие передачу исключения.

Повторная передача исключения (rethrow). Пустой оператор `throw` повторно передает объект исключения. Повторная передача возможна только из блока `catch` (обработчика) или из функции, прямо или косвенно вызываемой обработчиком. В результате будет повторно передан полученный ранее объект исключения.

Порядок выполнения конструкторов (constructor order). При не виртуальном наследовании части базовых классов строятся в том порядке, в котором они указаны в списке наследования класса. При виртуальном наследовании часть виртуального базового класса (классов) создается прежде любых других базовых классов. Они создаются в порядке расположения в списке наследования производного класса. Только самый последний производный тип может инициализировать виртуальный базовый

класс; списки инициализации конструктора этого базового класса, расположенные в промежуточных базовых классах, игнорируются.

Прокрутка стека (*stack unwinding*). Процесс выхода из функции при передаче исключения и перехода к поиску его обработчика. Локальные объекты, созданные перед передачей исключения, удаляются перед началом поиска соответствующего обработчика.

Пространство имен (*namespace*). Механизм, используемый для сбора всех имен, определенных в библиотеке или другом фрагменте программы, в единую область видимости. В отличие от других областей видимости языка C++, область видимости пространства имен может быть определена в нескольких частях. Пространство имен может быть открыто, закрыто и открыто вновь, причем в разных частях программы.

Псевдоним пространства имен (*namespace alias*). Синтаксис создания синонима для пространства имен имеет следующий вид: `namespace N1 = N;` где `N1` — это лишь другое имя пространства имен `N`. Пространство имен может иметь несколько псевдонимов, причем псевдонимы и реальное имя пространства имен могут использоваться попеременно.

Спецификация `noexcept`. Ключевое слово, обычно указывающее, передает ли функция исключение. Когда за списком параметров функции следует ключевое слово `noexcept`, за ним (необязательно) может следовать заключенное в скобки константное выражение, приводимое к типу `bool`. Если выражение отсутствует или возвращает значение `true`, функция не передает исключений. Если выражение возвращает значение `false` или у функции нет спецификации исключения, она может передать любое исключение.

Спецификация запрета передачи исключения (*nonthrowing specification*). Спецификация исключения, обещающая, что функция не будет передавать исключений. Если такая функция передаст исключение, то будет вызвана функция `terminate()`. К спецификаторам запрета передачи исключения относятся спецификатор `noexcept` без аргумента или с аргументом, возвращающим значение `true`, а также `throw()`.

Статический файловый объект (*file static*). Локальное для файла имя, которое было объявлено с использованием ключевого слова `static`. В языке C и версиях языка C++, выпущенных до появления стандарта, статические файловые объекты использовались для объявления таких объектов, которые применимы только в одном файле. Применение статических файловых объектов осуждено стандартом C++. Сейчас они заменены безымянными пространствами имен.

Функция `terminate()`. Библиотечная функция, вызов которой происходит в случае, когда переданное исключение либо так и не обработано, либо если оно было передано в обработчике исключений. Функция `terminate()` завершает выполнение программы.

